Residency: A Unified Theory of Fair Metering, Allocation, and Placement for Multi-Tenant LLM Serving

Anonymous Authors

Abstract

Two tenants of a shared LLM service can send the same number of tokens, pay the same token bill, and yet consume wildly different amounts of the one resource that is actually scarce. The reason is that the scarce resource in modern continuous-batched serving is not tokens but *memory occupied over time*: a long prompt holds KV cache for the entire generation that follows it, so equal token counts can hide a large disparity in held memory. We give this quantity a name and introduce *residency*—the KV memory a request occupies integrated over the time it occupies it (measured in token-seconds). A token meter is residency with the time axis projected away, and that lost axis is exactly where the unfairness hides. This paper presents a simple thesis: the right unit of account for fair serving is residency, not tokens—memory occupied over time, not tokens submitted.

In an idealized single-resource model we obtain a request's scalar residency in closed form—a request of prompt length P and output length D occupies $C_{\mathrm{KV}}(P, D) = P^2/2\pi + (PD + D^2/2)/\delta$ token-seconds, the area under its occupancy curve, with π, δ the idealized prefill and decode rates. This closed form is a *lens*, not a meter: it makes the structure of the scarce resource legible—prefill builds quadratically, prefill and decode separate—and makes the token-versus-residency separations provable in closed form, but the deployed mechanism never computes it. Enforcement is entirely backward-looking, draining each tenant's buckets on *measured* realized occupancy. But a real system may be disaggregated, tiered, and spread over heterogeneous instances, so residency is not a scalar but a *vector* across axes (prefill and decode HBM-time, KV-transfer-time, per-tier and per-instance occupancy-time). This residency vector is the central object of the paper, and our central structural result—a projection lemma—shows that the standard fairness meters are each this object with a coordinate collapsed: token-WFQ and VTC keep a token count but discard time, instantaneous KV quotas keep peak occupancy but discard its integral over time, and waiting-time accounting tracks a load-dependent symptom rather than the held resource. Each is a useful projection; none can by itself certify the guarantee tied to the axis it drops.

Using residency as the unit of account, we develop a theory of fair metering, allocation, and placement for LLM serving. As a *unit of fairness*, residency yields entitlements that cap each tenant's dominant share, with per-axis token buckets giving a two-sided guarantee—isolation (no tenant exceeds its share) and service (a backlogged tenant is not starved). As a *control variable*, it turns admission, continuation, preemption, routing, tiering, and prefill/decode disaggregation into a single online placement problem, solved by one urgency-weighted rule; priority and SLO tiers then decompose into a (share, burst, urgency) triple in which urgency cannot increase long-run share. We close by showing residency is physically measurable: on saturated workloads where token-fair scheduling looks fair yet skews residency several-fold, residency entitlements substantially reduce the skew and cut the conformant tenant's p95 time-to-first-token by 38–274× over the residency-blind baselines, at held utilization—and every residency-blind meter we test equalizes its own coordinate while leaving residency skewed, the strongest of them (request-count round robin) doing worse than no meter at all. These experiments validate the theory on a profiled-microbenchmark serving simulator across both its scalar projection and its full multi-axis vector, where a two-axis prefill/decode configuration—with snapshot DRF and stateful time-aware DRF as head-to-head baselines—confirms that collapsing the residency vector to a scalar forfeits dominant-share fairness and that the order of axis-combination and time-integration is itself load-bearing, and where a transfer-bandwidth experiment exhibits a stock/flow dissociation—a tenant holding half the memory yet monopolizing the spill link—that every occupancy meter, even the strongest time-aware one, is provably blind to. Together they confirm the abstraction is physically real, not merely an accounting convenience.

1 Introduction

Consider two tenants of a shared LLM service. Over an hour, each sends the same number of requests, the same number of input and output tokens, and receives an identical token bill. By every meter a production system commonly applies—request counts, input/output token counts, weighted token service—they have been treated identically and fairly. And yet one of them may have consumed several times as much of the one resource that is actually scarce. The first tenant sends short prompts; the second sends prompts that are occasionally enormous, and each long prompt holds KV cache resident through every step of the generation that follows it. Equal token counts, wildly unequal held memory. The "fair" scheduler cannot see the difference, because the quantity it meters has the relevant axis projected away.

This is the mirage at the center of the paper, and its cause is precise: *tokens are memory-residency with the time axis removed*. A token count records how much a request asked for, not how long it held the machine while being served. In continuous-batched serving the scarce resource is KV cache, and KV cache is held *over time*—so the physically honest unit of account is memory occupied integrated over time, which we call *residency*:

$$\text{residency} = \int \text{resident KV tokens} \times \, \mathrm{d}t \quad \text{(token-seconds)}.$$

Restoring the time axis that token meters discard is the entire move; everything else in the paper is working out its consequences.

LLM serving has made this unavoidable. Continuous batching, PagedAttention, chunked prefill, disaggregated prefill/decode, preemption, and live migration have made serving systems far more efficient, but they also make fairness harder to reason about. A tenant's request is no longer well described by "one job" or by a linear number of tokens. A long context can occupy scarce KV cache while many other tenants wait; a long output can keep an ever-growing KV footprint resident during every decode step; a burst of multi-turn sessions can consume a large fraction of GPU residency without looking unfair under a simple token counter.

A tenant should not merely be charged for how many tokens it submits. It should be charged for its residency—how much KV cache it occupies and for how long. This gives a physically grounded unit, KV-token-seconds, the area under a request's occupancy curve.

The resulting formulation is deliberately simple. Suppose a single GPU has KV capacity K tokens. Tenant i is assigned a memory-share entitlement $\omega_i \in [0, 1]$, where $\sum_i \omega_i \leq \eta < 1$ leaves headroom for fragmentation, burst absorption, and switching costs. Tenant i is therefore entitled to occupy, on average, an ω_i fraction of the GPU KV cache over time. Its accumulated usage is

$$A_i(t) = \int_0^t k_i(s) \, \mathrm{d}s,$$

where $k_i(s)$ is the number of tenant i's resident KV tokens at time s. Its deficit or bucket balance is

$$B_i(t) = \min\{B_i^{\max},\ B_i(0) + \omega_i K t - A_i(t)\}.$$

A tenant that runs below entitlement accumulates positive balance; a tenant that runs above entitlement drains it. The scheduler uses these balances to choose the KV-resident set. Thus, conserving residency today buys future scheduling priority.

This scalar picture is the right starting point but not the end of the story. Once serving is disaggregated (prefill and decode stress memory and bandwidth differently), tiered (KV state lives in HBM, host DRAM, or NVMe at different costs), and spread across heterogeneous instances, the bottleneck is no longer a single number but a *vector* of renewable residency resources. Our central contribution is to show that this vector is the unifying object of the entire problem, and to put it to work in three roles. We *meter* with it: the charge for a request is the time-integral of its placement. We *allocate* with it: fairness is dominant-resource fairness applied to that integral, and we prove a two-sided isolation/service guarantee on each tenant's dominant residency share. We *place* with it: admission, preemption, routing, tiering, and prefill/decode disaggregation become coordinates of one online placement problem solved by a single rule. Along the way we prove a projection lemma: each prior meter is a coordinate projection of this object that forfeits a specific guarantee. The scalar residency meter above is the one-axis projection of the same object.

Our contributions. This paper formulates residency-based serving and validates it empirically—both its scalar projection and its full multi-axis vector—giving each theoretical claim a falsifiable test on a profiled-microbenchmark serving simulator: the metering and fairness claims are measured here, the unified placement rule is run as the operative admission/preemption/eviction mechanism in both campaigns (so the master reduction's collapse of these controls is exercised, not just proved), and the remaining placement claims—routing, tiering, and the approximation bound—are stated with their empirical arm (a control-plane ablation requiring no oracle) scoped explicitly. The contributions are organized around one object and the four things we do with it, with the structural result that locates all prior work given its own pillar, and an experimental pillar that confirms the predicted effects from the single-axis special case through the axis-migration regime where the vector structure is decisive.

- **Residency as the unit of account.** We introduce residency—KV memory occupied integrated over time—as the unit for fair serving, and obtain its closed form in an idealized single-resource model, $C_{\mathrm{KV}}(P, D) = P^2/2\pi + (PD + D^2/2)/\delta$ (section 2), the area under a request's occupancy curve. The closed form is an *idealized lens*: it exposes the structure of residency and carries the closed-form separations of section 2, but the mechanism never computes it—the meter charges *measured* realized occupancy (definition 2), so every fairness and service guarantee runs on observed quantities and is independent of the formula's numerical accuracy. That the idealization is itself stable under realistic rate variation is recorded as model-robustness in section A. In a real system residency is a *vector* across prefill/decode, tier, transfer, and instance axes (definition 4); the scalar is its $m = 1$ projection (proposition 3). A tenant's entitlement ω_i is then directly its long-run share of that resource.

- **Several prior meters are special cases or projections of residency.** A projection lemma (lemma 1) locates prominent prior meters relative to our object: the decode-only KV-token-time of Justitia is an exact limiting case (proposition 4), while token-WFQ/VTC (proposition 5) and static DRF (proposition 6) are lossy projections that collapse the time axis and the trajectory respectively, each forfeiting a guarantee the full meter provides—including a distributional separation that does not rely on degenerate unit-length requests (proposition 2). This locates the prior literature inside the object rather than against it.

- **Fairness on residency.** Entitlements cap each tenant's *dominant* residency share via a DRF-inspired time-integrated dominant-share entitlement, enforced by per-axis token buckets with a two-sided guarantee: rate-plus-burst isolation (theorem 2) and backlog service/no-starvation (theorem 3), derived from network-calculus arrival curves. Priority and SLO tiers decompose into a (share, burst, urgency) triple, with an urgency-invariant isolation theorem (theorem 4) and a share-neutrality result (proposition 10): self-declared priority reorders a tenant's own requests but cannot increase its long-run share.

- **Placement as unified control.** A master reduction (theorem 6) makes admission, continuation, preemption, routing, tiering, and prefill/decode disaggregation one online placement problem over the vector. The per-epoch problem is NP-hard; a single urgency-weighted density rule yields a greedy selector with an unconditional $\frac{1}{m+1}$ guarantee on m axes by reduction to multidimensional knapsack (theorem 8), sharpening to $\frac{1}{2}$ in the common single-binding-axis regime (section 4.1); from this rule scheduling, routing, preemption, and KV eviction (including tiered demotion) are read off across a cluster/instance two-plane decomposition, eviction dual to admission (section 4.3).

- **Empirical validation, scalar and vector.** On a discrete-event serving simulator with profiled-microbenchmark latency (Llama 3.1-8B/H100), under production-realistic demand asymmetries, residency entitlement (KVtime) reduces the residency-share skew 2.2–2.9× and the conformant tenant's p95 TTFT by 38–274× over residency-blind meters (section 5); on the prompt-asymmetric workload it does so at the *highest* goodput of all eight schedulers, and where its aggregate goodput is lower (under rate abuse) the reduction is the bounded-share guarantee throttling the abusive tenant while lifting the conformant tenant's coverage from ∼0.52 to ∼0.96 (section 5.4). An eight-scheduler wall makes the projection lemma concrete: every prior meter—including the two closest prior fairness designs, Justitia and Equinox, run as meters under our mechanism—equalizes its own coordinate yet leaves residency skewed, with request-count round robin worse than no meter at all; a strict instantaneous KV quota drops exactly the heavy-tail fraction the analysis predicts while KVtime drops none; and the service bound (theorem 3) holds across the bucket-depth sweep. A two-axis prefill/decode campaign then validates the residency *vector* itself: collapsing it to a

scalar inflates dominant-share disparity $\sim 5\times$, the regulated schedulers coincide exactly under constant load (proposition 8), and under axis migration vector KVtime delivers materially tighter per-window isolation than stateful DRF run on occupancy vectors (SDRF)—the strongest time-aware multi-resource baseline, included head-to-head in every vector experiment—establishing that the integrate-then-combine ordering is load-bearing, not stylistic. A transfer-bandwidth experiment makes the stock/flow distinction a measured fact: a stowaway tenant holds $0.51\times$ the HBM of a quiet tenant yet drives $1.83\times$ its share of the spill link, and proposition 9 shows every occupancy meter—naive or time-aware—is structurally blind to it (section 5.3). Real-system feasibility of low-overhead occupancy metering—the one quantity the mechanism does depend on—we leave to future work.

Every experiment is stated with the exact hypothesis it confirms or falsifies, and section 5 reports the measured results, scalar and vector—so the boundary between what is established here and what we leave to future work is explicit throughout the paper.

2 Residency as the Unit of Account

We put the time axis back in the simplest setting where it can be seen cleanly—one resource, continuous time—obtain the closed-form residency there as an idealized lens on the resource's structure, and then state the general object of which it is the single-axis case. The lens makes the structure legible and the separations provable; the meter itself, here and throughout, charges measured occupancy. This is Pillar 1: what we meter.

We begin with the cleanest possible model. Time is continuous. There is one GPU with KV capacity K tokens. Prefix caching is ignored for now. A request has prompt length $P \geq 0$ and output length $D \geq 0$.

The GPU can prefill at throughput π tokens/s and decode at throughput δ tokens/s. Thus, processing an infinitesimal prefill token mass $\mathrm{d}p$ takes $\mathrm{d}p/\pi$ seconds; processing an infinitesimal decode token mass $\mathrm{d}d$ takes $\mathrm{d}d/\delta$ seconds.

During prefill, after p prompt tokens have been processed, p tokens are resident in KV. During decode, after d output tokens have been generated, $P + d$ tokens are resident in KV.

Definition 1 (KV-time)**.** *The KV-time consumed by a request is the area under its KV-residency curve:*

$$C_{\mathrm{KV}}(P, D) = \int_0^{T(P,D)} k(t)\,\mathrm{d}t,$$

where $k(t)$ is the number of resident KV tokens of the request at time t.

Theorem 1 (Closed-form KV-time price)**.** *In the continuous-time model above, the KV-time consumed by a request with prompt length P and output length D is*

$$\boxed{C_{\mathrm{KV}}(P, D) = \frac{P^2}{2\pi} + \frac{PD + D^2/2}{\delta}.}$$

Proof. During prefill, when p prompt tokens are resident, processing the next infinitesimal mass $\mathrm{d}p$ takes $\mathrm{d}p/\pi$ seconds. The contribution to KV-time is $p\,\mathrm{d}p/\pi$. Hence

$$C_{\mathrm{pre}}(P) = \int_0^P p\frac{\mathrm{d}p}{\pi} = \frac{P^2}{2\pi}.$$

During decode, after d output tokens are generated, the request has $P + d$ resident KV tokens. Generating $\mathrm{d}d$ output tokens takes $\mathrm{d}d/\delta$ seconds. Therefore

$$C_{\mathrm{dec}}(P, D) = \int_0^D (P + d)\frac{\mathrm{d}d}{\delta} = \frac{PD + D^2/2}{\delta}.$$

Adding the two phases proves the claim. □

This closed form is a result of the idealization above, not a restatement of a known cost. Two features are what make it the general scalar residency rather than a special case. First, it carries the *prefill build-up term* $P^2/2\pi$: the cost of holding a growing KV cache *during* prefill, as the prompt is processed token by token. A meter that sums occupancy only over generated (decode) tokens—charging $PD + D^2/2$ in iteration units, as memory-centric application schedulers do—cannot see this term, because the prefill phase produces no output tokens to sum over. Second, it separates the two throughputs π and δ, so prefill and decode occupancy are converted to time at their own physical rates. Sending the prefill rate to infinity ($\pi \to \infty$) makes the build-up term $P^2/2\pi$ vanish, and normalizing the decode rate to unit ($\delta = 1$) leaves exactly $PD + D^2/2$—the decode-only KV-token-time. Our scalar residency is therefore the strict generalization—prefill-aware and rate-separated—of which the decode-only KV-token-time is the $\pi \to \infty$, $\delta = 1$ specialization (sections 2.4 and 6).

Remark 1 (Why the PD term matters)**.** *The term PD/δ is the prompt-residency term: every decoded token carries the entire prompt in memory. This term is invisible to output-token counters and only indirectly visible to input-token counters.*

2.1 Token accounting is not resource-faithful

Having derived the residency price C_{KV}, we can already see why it is not interchangeable with the meter every production stack actually uses—the token count—even in the cleanest model. Token-based scheduling uses a cost like

$$C_{\mathrm{tok}}(P, D) = aP + bD.$$

KV-time uses $C_{\mathrm{KV}}(P, D)$. These are fundamentally different.

Proposition 1 (Token fairness can be arbitrarily unfair in KV-time)**.** *Fix prefill-only requests with $D = 0$. For any integer $N \geq 1$, compare two tenants that each submit N prompt tokens in total. Tenant A submits one request of length N. Tenant B submits N requests of length 1. Token accounting charges both tenants N tokens. KV-time charges*

$$C^A_{\mathrm{KV}} = \frac{N^2}{2\pi}, \qquad C^B_{\mathrm{KV}} = N \cdot \frac{1}{2\pi} = \frac{N}{2\pi}.$$

Thus $C^A_{\mathrm{KV}}/C^B_{\mathrm{KV}} = N$, which is unbounded as $N \to \infty$.

Proof. The calculation follows directly from the closed-form price with $D = 0$. Token accounting is additive in tokens and therefore assigns both tenants cost N. KV-time is quadratic in request length during prefill, so batching the same tokens into one long resident request costs N times more memory-time than splitting them into unit requests. □

This separation does not imply that one-token requests are operationally free; real systems have fixed overheads. It shows that token count is not order-equivalent to memory-time, even in the cleanest model.

The unit-length construction is deliberately extreme. The separation does not depend on it: it persists for realistic, non-degenerate request-length distributions, because KV-time is convex (quadratic) in length while token cost is linear.

Proposition 2 (Distributional separation without degenerate requests)**.** *Let two tenants submit the same total token volume and the same number of requests, drawing prefill-only request lengths from distributions $\mathcal{P}_A$ and $\mathcal{P}_B$ with equal mean $\bar{P}$. Token accounting charges both the same expected cost $\propto \bar{P}$. KV-time charges expected cost $\frac{1}{2\pi}\mathbb{E}[P^2] = \frac{1}{2\pi}(\bar{P}^2 + \mathrm{Var}\, P)$. Hence the KV-time ratio between the tenants equals*

$$\frac{\bar{P}^2 + \mathrm{Var}_A}{\bar{P}^2 + \mathrm{Var}_B},$$

which exceeds any fixed constant c once $\mathrm{Var}_A \geq c(\bar{P}^2 + \mathrm{Var}_B) - \bar{P}^2$. Two tenants identical in token volume, request count, and mean length thus differ unboundedly in memory-time purely through length variance.

Proof. Token cost is linear, so equal mean and count give equal token charge. For prefill-only requests $C_{\mathrm{KV}} = P^2/2\pi$, so the expected KV charge is $\frac{1}{2\pi}\mathbb{E}[P^2]$, and $\mathbb{E}[P^2] = \bar{P}^2 + \mathrm{Var}\, P$ by definition of variance. Taking the ratio and solving the stated inequality for Var_A gives the bound; since variance is unbounded above for fixed mean, the ratio is unbounded. □

This is the operationally meaningful version: a tenant whose workload has the same average prompt length but heavier-tailed length variation imposes strictly more KV pressure, and a linear token meter cannot see it.

Definition 2 (Nominal vs. realized KV-time)**.** *The closed form $C_{\mathrm{KV}}(P, D)$ above is the* nominal active-service *KV-time: the area under the occupancy curve of a request that progresses continuously through prefill and then decode. In a real system a request can be* resident without progressing*—waiting between prefill and decode, stalled behind other work, or held in memory across a preemption gap. Its* realized *KV-time is the area over its entire residency,*

$$\widetilde{C}_r = \int_{resident} k_r(t)\,\mathrm{d}t \;\geq\; C_{\mathrm{KV}}(P, D),$$

the inequality holding because resident-but-idle intervals only add nonnegative area. The scheduler charges the measured *realized area online—buckets drain on the observed occupancy $k_i(t)$, not on the formula—so the fairness guarantees of section 3.1 use $\widetilde{C}_r$ directly. Enforcement is therefore entirely* backward-looking*: a tenant's bucket balance is an integral of its own past measured occupancy, and admission, preemption, and placement decisions read that accumulated balance. Nothing in the loop estimates a request's future residency, so the closed form C_{KV} is never on the critical path. Its role is the one it is uniquely good at: exposing the structure of residency and supplying the clean functional form the separations of section 2 are proved on.*

Remark 2 (Charging idle residency does not tax a tenant for the system's own choices)**.** *Charging* realized *occupancy raises a fair question: if a request is held resident while stalled—waiting between prefill and decode, or pinned to satisfy another tenant's deadline—is its owner unfairly billed for memory it is not actively using? The answer is that idle residency is real scarcity (the held block is unavailable to everyone else regardless of whether its owner is making progress), so it is correct to meter it; but the mechanism is built so that* unproductive *pinning is something the scheduler avoids rather than charges for.* GreedyKV *(section 4.1) is work-conserving and evicts or demotes a stalled request to a slacker tier rather than holding it idle in the scarce tier: a request with no imminent progress and low score is precisely the one freed when space is contended, so it stops accruing residency on the binding axis instead of being taxed on it. When the platform* does *deliberately keep a tenant resident for the system's benefit (e.g. pinning its KV so another tenant hits a deadline), that is a policy decision whose cost should be borne by the platform or the beneficiary, not silently charged to the pinned tenant; our accounting makes that cost* visible *as residency on the pinned tenant's axis, which is the prerequisite for a credit or exemption policy, rather than hiding it as a token meter would. Metering the idle interval is thus what surfaces the externality; the eviction/demotion behavior of the placement rule is what keeps it from accumulating in the first place.*

Remark 3 (Shared prefixes: attribution that conserves)**.** *Prefix caching makes one physical KV copy serve many live requests, possibly across tenants, and a meter must say who pays. Our attribution follows the cause of the residency, at two granularities.* Across instances*, a prefix replicated on several GPUs is several independent residencies: each copy exists because requests carrying that prefix were routed there, so each copy is charged on its instance-axis to the tenants whose requests materialized it, at the times they hold it.* Within an instance*, a shared block's residency at each instant is split among the tenants whose live requests currently reference it (reference-count split: $1/k$ each when k tenants share). This is the unique attribution under which per-tenant occupancies sum exactly to physical occupancy, preserving the conservation identity $\sum_i k_i^s(t) = k_{\mathrm{total}}^s(t)$ that our experimental apparatus verifies in every cell—full charging of each sharer would over-count the cache. It also sets the right price signal: sharing a prefix* reduces *every sharer's residency charge, because shared state genuinely consumes less aggregate scarcity per request—the metering-side counterpart of the locality benefit that DLPM-style schedulers pursue [12], which otherwise enters our model through switching costs. Finally, blocks* retained speculatively after *all referencing requests complete are referenced by no tenant; they are platform-policy residency, handled exactly as deliberate pinning above: under contention the density rule evicts them first (zero progress density), so no one pays for state that does not survive scarcity, and under slack their cost is platform-visible rather than smeared across tenants.*

2.2 From scalar to vector: the general residency meter

The closed form above is the right primitive for a single, undifferentiated memory pool. Real serving is not undifferentiated: prefill and decode stress memory and bandwidth differently; KV state can live in HBM,

host DRAM, or NVMe at different access costs; and a cluster offers heterogeneous instances. In each case the bottleneck splits into several distinct *renewable memory-time resources*, and residency becomes not a scalar but a *vector* across those axes. This is the object the rest of the paper meters, allocates, and places; the scalar closed form is its single-axis shadow.

The vector meter is abstract by design, but its axes are not hypothetical—each corresponds to a resource that production LLM serving stacks already manage explicitly. We ground the axes in current systems and state the time-integral each one meters. Two kinds of axis arise, and the distinction matters: *capacity* axes, which are scarce in space and metered by occupancy integrated over time (residency); and *bandwidth* axes, which are scarce in throughput and metered by transferred volume (transfer rate integrated over time). They are different integrals of different instantaneous quantities, and conflating them loses a contention mode that bites in practice.

Capacity axes (residency, $\int k_i^r(t)\,\mathrm{d}t$). The memory hierarchy holding KV state is tiered, and modern stacks place blocks across tiers explicitly. A representative hierarchy spans GPU HBM ($\sim$3.3 TB/s), host DRAM ($\sim$60 GB/s), local NVMe, and network/remote storage, each a distinct capacity at a distinct access cost [7, 6]. Systems that span these tiers—LMCache and Mooncake's CPU/DRAM/SSD KV pools [7, 6], NVLink-domain offload into peer-GPU HBM (Aqua [35])—are exactly choosing per-tier residency. Each tier is a capacity axis with its own C^r, metered by the occupancy-time $\int k_i^r(t)\,\mathrm{d}t$ of tenant i's blocks resident in that tier. *Prefill/decode disaggregation* adds capacity axes of a different flavor: because prefill is compute-bound and decode is memory-bandwidth-bound, production deployments run them on separate GPU pools (DistServe [5], Mooncake [6]), so prefill-HBM-time and decode-HBM-time are distinct axes a request occupies in disjoint phases. *Multi-instance routing* adds one capacity axis per device: HBM on GPU g is its own axis C_g^r, since capacity is per-device and a request resident on one GPU is not resident on another. Production routing layers make this axis explicit—llm-d performs KV-cache- and prefix-aware routing across a heterogeneous, hardware-topology-aware pool of prefill/decode pods [10], choosing which device's HBM a request occupies. In our four-tier instantiation, the two genuinely space-scarce tiers (HBM, DRAM, per device) are capacity axes; the deeper tiers (local and network storage) have effectively unbounded capacity for our purposes and enter not as capacity axes but through the cost of moving data into and out of them—which is the next kind of axis.

Bandwidth axes (transfer, $\int \tau_i^r(t)\,\mathrm{d}t$). Moving KV between tiers or between instances consumes a shared, throughput-scarce interconnect: NVLink/NVSwitch within a node, PCIe to host and storage, and RDMA over the network between nodes. This fabric is a first-class resource in current systems—Mooncake pools "cache capacity *and* transfer bandwidth" over a 100–400 Gbps RDMA network it notes is "comparable to memory bandwidth" [6], and NVIDIA's NIXL and the Dynamo KV block manager exist specifically to move KV across GPUs and nodes at wire speed [8, 9]. A transfer axis is metered not by occupancy but by transferred volume—the time-integral $\int \tau_i^r(t)\,\mathrm{d}t$ of tenant i's instantaneous transfer rate $\tau_i^r(t)$ on link r, i.e. total bytes moved, with the link's bandwidth as its capacity C^r.

Why transfer is its own axis and not inferable from residency. It is tempting to fold transfer into residency on the grounds that resident KV is what gets moved. This fails because residency is a *stock* (occupancy) and transfer is a *flow* (rate), and the two are not functions of one another. Two tenants with identical HBM residency can differ arbitrarily in bandwidth—one re-reading or migrating its resident KV constantly, the other holding it idle—and, more sharply, a tenant that *spills aggressively* keeps measured residency *low* while driving transfer *high*, so a residency-only meter rates the heaviest user of the contended fabric as the lightest. When the fabric is uncontended (high-bandwidth NVLink, occasional migration), transfer is well modeled as a per-request *switching cost* E_r, R_r on the capacity axes (definition 3), and the four-tier picture collapses to two capacity axes plus switching cost. When the fabric is contended—steady cross-tier or cross-node streaming, as in long-context offload and disaggregated serving at scale—transfer must be promoted to its own bandwidth axis, metered by bytes moved, so that cross-tenant contention for the link is governed by the same dominant-share entitlement as memory. The vector meter accommodates both: a switching cost in the uncontended regime, a full axis when bandwidth binds. This stock-versus-flow split is a second, independent reason the meter must be a vector—not only do prefill and decode occupy different capacity axes, but capacity and bandwidth are different integrals that a single scalar cannot represent.

2.3 Resources, placements, and the vector meter

Fix a finite set of *memory-time resources* (axes) $r \in \{1, \dots, m\}$. Each axis is renewable at capacity rate $C^r > 0$: over any window of length Δ it offers $C^r \Delta$ units of occupancy-time. Examples: prefill-HBM-time and decode-HBM-time (PD disaggregation); HBM-time, host-DRAM-time, NVMe-time, and KV-transfer-time (tiering); and per-instance HBM-time on a heterogeneous cluster (routing). Several effects coexist by taking the product set of axes.

A live request r at time t is executed through a *placement* $\boldsymbol{\varphi}_r(t) \in \mathbb{R}^m_{\geq 0}$, the vector of resource occupancies it induces at that instant. The platform's control is the placement field $\boldsymbol{\varphi}(t) = (\boldsymbol{\varphi}_r(t))_{r \text{ live}}$, chosen online subject to per-axis feasibility and switching costs.

Definition 3 (Feasible placement field)**.** *A placement field is feasible at t, written $\boldsymbol{\varphi}(t) \in \mathcal{F}(t)$, if for every axis r the aggregate occupancy obeys $\sum_{live} \varphi^r_\cdot(t) \leq C^r$, and if each live request's placement lies in its admissible set $\Phi_r(t)$ (the placements that execute it given cached prefixes, tier residency, and instance assignment). Moving a request between admissible placements incurs a switching cost (eviction E, resume/recompute R, or cache-move).*

Definition 4 (Memory-time resource vector)**.** *Tenant i's memory-time consumption over $[0, T]$ is the vector*

$$\mathbf{M}_i(T) = \int_0^T \mathbf{k}_i(t)\, \mathrm{d}t \in \mathbb{R}^m_{\geq 0}, \qquad k^r_i(t) = \sum_{r' \in \text{ tenant } i} \varphi^r_{r'}(t),$$

i.e. the time-integral of tenant i's aggregate placement on each axis. Its normalized share on axis r is $s^r_i(T) = M^r_i(T)/(C^r T)$, and its dominant memory-time share *is $s_i(T) = \max_r s^r_i(T)$.*

Remark 4 (The axis set is open-ended)**.** *The vector is defined by what an axis* is*—a metered resource with occupancy-over-time semantics and a capacity (stock) or budget (flow)—not by the particular axes enumerated above. When a different resource becomes the binding one, it joins the vector as an axis rather than breaking the abstraction: host-CPU orchestration time, shown by Blink to be a critical-path bottleneck that warrants removing the CPU from steady-state serving entirely [34], is an admissible flow axis (orchestration-seconds against a budget), as are SmartNIC buffers or any other occupancy-metered stage. The scarce resource is context-dependent; the framework's claim is that whatever it is, fairness on it is residency on the corresponding axis, with the same bucket guarantees.*

Notational convention. Throughout the paper, *axis* indices appear as superscripts and *request* indices as subscripts: C^r, k^r_i, and M^r_i name quantities of a resource axis r, while $\hat{w}_r$, $\boldsymbol{\varphi}_r$, and E_r name quantities of a candidate request r. The position of the index, not the letter, disambiguates; where both appear together we write φ^s_r for request r's occupancy on axis s.

The charge for a completed request is the time-integral of its own placement, $\int \boldsymbol{\varphi}_r(t)\, \mathrm{d}t$, aggregated over axes by a posted weight vector $\mathbf{c} \succ 0$ (the relative price of each memory-time resource). Scalar KV-time is the case $m = 1$.

Proposition 3 (Scalar KV-time is the one-dimensional projection)**.** *With $m = 1$, a single HBM axis of capacity K, and the phase-pure trajectory of definition 1, the vector meter reduces to $M_i(T) = A_i(T) = \int_0^T k_i(t)\, \mathrm{d}t$ and a completed request's charge is exactly $C_{\mathrm{KV}}(P, D) = P^2/2\pi + (PD + D^2/2)/\delta$. The entitlement ω_i caps the dominant share s_i.*

Proof. Immediate from definition 4: a single axis makes $\mathbf{k}_i$ scalar, the integral is A_i, and the per-request integral is the area computed in the closed-form theorem. With one axis $\max_r s^r_i = s^1_i = A_i/(KT)$, and the bucket of section 3.1 enforces $\limsup A_i/T \leq \omega_i K$, i.e. $s_i \leq \omega_i$. □

2.4 Special cases and projections: how residency relates to prior meters

Pillar 2: where prior meters sit. Before allocating or placing, we locate the existing meters as coordinate collapses of the residency object—one an exact limiting case, the others lossy projections that drop an axis and forfeit the guarantee tied to it. This is the structural result that lets us treat prior work as projections to place inside the object rather than competitors to argue against.

Several prominent meters in the literature stand in a precise relation to residency. One is a *limiting case*—recovered exactly by specializing the residency charge; two others are *projections*—lossy collapses that drop a coordinate and thereby forfeit a guarantee residency provides. We state each as a short proposition. We do not claim to subsume *every* conceivable meter, only to locate these prominent ones relative to our object.

Proposition 4 (Limiting case: decode-only KV-token-time, Justitia [18])**.** *Justitia meters the accumulative KV-cache occupation $pd + d^2/2$ (KV-token-time) to rank applications by fair completion order. This is the residency charge $C_{\mathrm{KV}}(P, D) = P^2/2\pi + (PD + D^2/2)/\delta$ in the rate limit $\pi \to \infty$, $\delta = 1$:*

$$C_{\mathrm{KV}}(P, D) \xrightarrow{\pi\to\infty,\ \delta=1} PD + D^2/2.$$

Proof. As $\pi \to \infty$ the build-up term $P^2/2\pi \to 0$; with $\delta = 1$ the remainder is $PD + D^2/2$. □

What it loses. Two things. It discards the prefill build-up entirely (the $\pi \to \infty$ limit zeroes $P^2/2\pi$), so it does not charge for memory held *during* prefill—the cost that grows with prompt length. And it is a single scalar, so it cannot express dominant-resource fairness across differentially scarce axes (prefill/decode, tier, transfer); it is the $m = 1$ case of definition 4 (proposition 3). Residency recovers its cost exactly and restores both.

Proposition 5 (Projection: token-WFQ / VTC [17] is the time collapse)**.** *A token meter charges $C_{\mathrm{tok}}(P, D) = aP + bD$, a function of endpoints invariant to how long KV remains resident. It is the residency meter with the time axis collapsed, and it cannot bound residency: at fixed token charge the residency ratio between tenants is unbounded (proposition 2).*

Proof. C_{tok} depends only on (P, D), hence is invariant under time-reparameterization of the occupancy trajectory; proposition 2 exhibits unbounded $\int k_i$ at fixed (P, D), so no rate-plus-burst bound of the form theorem 2 can follow from it. □

What it loses. The duration axis. As section 2.1 already showed, two tenants with identical token charge can differ unboundedly in residency through length variance alone; the token meter is blind to it. VTC is the cleanest instance: it meters weighted token service and forgets the time those tokens are held.

Proposition 6 (Projection: static DRF [24] is the time-collapse of dominant share)**.** *Dominant resource fairness applied to instantaneous occupancy vectors equalizes the snapshot dominant share $\max_r k_i^r(t)/C^r$, not its time-integral. It is residency with the trajectory replaced by a snapshot; it coincides with the residency entitlement only when occupancy is time-invariant (proposition 8), and diverges whenever occupancy varies over time.*

Proof. Under constant occupancy $\mathbf{k}_i(t) \equiv \mathbf{k}_i$ the share k_i^r/C^r is time-independent, so snapshot DRF equals the time-integrated cap (proposition 8). For time-varying occupancy the snapshot and the integral differ, so the snapshot does not meter residency. □

What it loses. The time axis, while keeping the resource axes. DRF is multi-resource but instantaneous, so it cannot certify a *long-run* dominant-share entitlement; a tenant within its snapshot share can still dominate residency by holding that share longer. Time-aware DRF variants—finish-time fairness [25] and stateful DRF [26]—add time but integrate the wrong quantity (completion time, and time-averaged share, respectively, not the time-integral of occupancy); section 6 draws the distinction.

Lemma 1 (Projection lemma)**.** *Each of the prior meters above is a coordinate collapse of the vector residency meter that forfeits a residency guarantee: token-WFQ/VTC collapses the time axis (proposition 5, losing the rate-plus-burst bound of theorem 2); static DRF and the instantaneous KV quota collapse duration (proposition 6, losing the long-run dominant-share entitlement); and any scalar memory-time meter collapses the resource axes to $m = 1$ (proposition 3, losing dominant-resource fairness). The decode-only meter of proposition 4, though its cost is recovered exactly, is such a scalar collapse.*

Taken together, propositions 4 to 6 locate these prominent prior meters inside our object: one as an exact limiting case, the rest as lossy projections, each forfeiting a guarantee the full meter provides. This closes the metering pillar.

3 Fairness on Residency

Pillar 3: what a fair allocation of residency is. We first turn request residency into a per-tenant share and bucket (the single-axis picture), then state the entitlement and its two-sided guarantee on the full vector, and finally show how priority and SLO tiers fit the same object without being able to steal share.

Let $\mathcal{I}$ be the set of tenants. Tenant i receives a memory-share entitlement

$$\omega_i \in [0,1], \qquad \sum_{i\in\mathcal{I}} \omega_i \le \eta < 1.$$

The parameter η is the usable-capacity factor. It leaves headroom for fragmentation, scheduling granularity, switching costs, and burst absorption.

Let $k_i(t)$ be the total resident KV tokens belonging to tenant i at time t. Tenant i's accumulated KV-time usage is

$$A_i(t) = \int_0^t k_i(s)\,\mathrm{d}s.$$

Its nominal entitlement over $[0,t]$ is $\omega_i K t$. Define the unclipped deficit

$$\Delta_i(t) = \omega_i K t - A_i(t).$$

Positive Δ_i means the tenant is below its memory-time entitlement; negative Δ_i means it is ahead of entitlement.

Definition 5 (KV-time bucket)**.** *Tenant i has a burst bucket of size $B_i^{\max} \ge 0$ and overdraft allowance $H_i \ge 0$. The clipped balance $B_i(t)$ evolves as*

$$B_i(t) = \min\{B_i^{\max},\ B_i(0) + \omega_i K t - A_i(t)\},$$

with the enforcement rule that tenant i may consume KV-time only while $B_i(t) > -H_i$, except possibly on explicitly marked best-effort slack capacity.

When $H_i = 0$, a tenant cannot spend beyond its bucket. When $H_i > 0$, H_i is a bounded debt tolerance.

3.1 Scalar fairness: rate-plus-burst on a single pool

The bucket gives a simple service-curve guarantee, the time-integrated analogue of token-bucket networking: a tenant is regulated by a leaky bucket whose "rate" is its entitled occupancy $\omega_i K$ and whose "burst" is the bucket depth $B_i^{\max} + H_i$, measured in KV-token-seconds. We do not prove this twice. It is the single-axis case of the general per-axis isolation theorem (theorem 2), and we state it precisely as corollary 1 once the general bound is in hand. The reader content with the scalar picture can read that corollary now; the only fact used in the rest of this section is its consequence $\limsup_{T\to\infty} A_i(T)/T \le \omega_i K$—a tenant cannot, in the long run, occupy more than its entitled share of KV memory.

3.2 Headroom and fragmentation

Fragmentation means that raw capacity K may not be fully usable at every instant. We model this by an effective feasible capacity lower bound.

Definition 6 (Fragmentation loss)**.** *A feasibility process $\mathcal{F}(t)$ has fragmentation loss at most $(1-\eta)K$ if every collection of tenant demands whose aggregate KV occupancy is at most ηK can be placed, possibly after allowed preemptions/resumptions.*

Proposition 7 (Headroom makes entitlements feasible)**.** *If $\sum_i \omega_i \le \eta$ and fragmentation loss is at most $(1-\eta)K$, then the aggregate nominal entitled occupancy $\sum_i \omega_i K$ is feasible under the effective capacity lower bound.*

Proof. The aggregate nominal entitlement is $\sum_i \omega_i K \le \eta K$. By the definition of fragmentation loss, any demand vector with aggregate occupancy at most ηK is placeable. Hence the entitlement vector is feasible. □

Thus entitlements should be interpreted as shares of *usable* memory-time, not necessarily shares of raw physical KV capacity.

3.3 Fairness: time-integrated Dominant Resource Fairness

Entitlements generalize from a scalar share to a cap on dominant memory-time share. Tenant i is allotted ω_i with $\sum_i \omega_i \le \eta < 1$.

Definition 7 (Time-integrated dominant-share entitlement (TIDSE))**.** *Each tenant is assigned a dominant-share entitlement ω_i with $\sum_i \omega_i \le \eta < 1$, and the mechanism enforces $s_i \le \omega_i$ for all i via per-axis token buckets. This is a DRF-*inspired *entitlement cap on the time-integrated dominant share—an enforceable* regulator*, not an allocation rule. It is the object all our fairness theorems concern: theorems 2 and 3 prove two-sided guarantees (isolation and service) on this cap. An operator configures only the entitlements $\{\omega_i\}$ (with headroom η, bursts β, overdrafts H_i): under symmetric entitlements $\omega_i = \omega$ the cap drives realized dominant shares toward equality up to bucket slack, exactly the convergence the β-sweep of section 5 exhibits ($\rho_{\mathrm{mt}} \to 1$ as β grows); under deliberately unequal ω_i it targets proportional shares. The cap converges to the configured target shares at a rate the burst depth controls—the property an operator actually needs to provision against. An idealized allocation-rule counterpart (leximin time-integrated DRF), and the precise sense in which it recovers classical DRF, are of theoretical interest only and are developed in section C; no result in this section depends on them.*

Theorem 2 (Per-axis rate-plus-burst isolation)**.** *Suppose each tenant i is regulated on each axis r by a leaky bucket with rate $\omega_i C^r$, depth $B_i^{r,\max}$, and overdraft H_i^r. Then for every axis r and interval $[t_1, t_2]$,*

$$M_i^r(t_2) - M_i^r(t_1) \le \omega_i C^r (t_2 - t_1) + B_i^{r,\max} + H_i^r,$$

and consequently the long-run dominant share obeys $\limsup_{T\to\infty} s_i(T) \le \omega_i$.

Proof. The whole proof is one accounting identity for a single bucket, applied to each axis and then maximized over the finite axis set. Fix an axis r. Model tenant i's bucket on r as a reflected accumulator: it refills at rate $\omega_i C^r$, drains at the measured occupancy rate $k_i^r(t) \ge 0$, saturates at depth $B_i^{r,\max}$ (refill above this is discarded), and forbids consumption once the balance reaches the overdraft floor $-H_i^r$. Let $L \ge 0$ be the refill credit lost to saturation over $[t_1, t_2]$. Conservation of the bucket gives the exact identity

$$B_i^r(t_2) = B_i^r(t_1) + \omega_i C^r (t_2 - t_1) - \big(M_i^r(t_2) - M_i^r(t_1)\big) - L,$$

since what enters (refill) minus what leaves (consumption $M_i^r(t_2) - M_i^r(t_1)$ plus discarded credit L) is the change in balance. Rearranging for consumption,

$$M_i^r(t_2) - M_i^r(t_1) = \omega_i C^r (t_2 - t_1) + \big(B_i^r(t_1) - B_i^r(t_2)\big) - L.$$

Now bound the two correction terms by the bucket's own limits: $B_i^r(t_1) \le B_i^{r,\max}$ (saturation cap), $B_i^r(t_2) \ge -H_i^r$ (overdraft floor), and $L \ge 0$. Hence $B_i^r(t_1) - B_i^r(t_2) \le B_i^{r,\max} + H_i^r$ and

$$M_i^r(t_2) - M_i^r(t_1) \le \omega_i C^r (t_2 - t_1) + B_i^{r,\max} + H_i^r,$$

the displayed per-axis bound. This is exactly a $(\omega_i C^r,\ B_i^{r,\max} + H_i^r)$ arrival curve in the sense of deterministic network calculus [27, 28]: rate measured in occupancy, burst in occupancy-time. Finally divide by $C^r(t_2 - t_1)$, set $t_1 = 0$, and let $T \to \infty$: the burst terms are constants, so $\limsup_T s_i^r(T) \le \omega_i$ for each axis r. Taking the maximum over the finite axis set, $\limsup_T s_i(T) = \limsup_T \max_r s_i^r(T) \le \omega_i$. □

The single-axis case is worth stating on its own, because it is the regime of a single, undifferentiated KV pool and it is what the scalar mechanism of sections 2 to 3.1 enforces.

Corollary 1 (Burst-bounded scalar residency)**.** *With one axis ($m = 1$, $C^1 = K$, $M_i^1 = A_i$), theorem 2 reads: for any interval $[t_1, t_2]$,*

$$A_i(t_2) - A_i(t_1) \le \omega_i K (t_2 - t_1) + B_i^{\max} + H_i, \qquad \textit{hence} \qquad \limsup_{T\to\infty} \frac{A_i(T)}{T} \le \omega_i K.$$

A tenant is regulated by a $(\omega_i K,\ B_i^{\max} + H_i)$ leaky bucket: long-run rate $\omega_i K$, burst depth $B_i^{\max} + H_i$.

Proof. Set $m = 1$ in theorem 2: the single axis has capacity $C^1 = K$, occupancy-time $M_i^1 = A_i$, depth $B_i^{1,\max} = B_i^{\max}$, and overdraft $H_i^1 = H_i$. Substituting these names into the theorem's bound gives the statement; the dominant share over one axis is the single share, so the rate bound is the theorem's $\limsup_T s_i(T) \le \omega_i$ rewritten as $\limsup_T A_i(T)/T \le \omega_i K$. □

Proposition 8 (Static and single-axis limits of the enforced cap)**.** (i) *If $m = 1$, the enforced TIDSE cap reduces to the scalar dominant-share entitlement of section 3.1 ($s_i = A_i/(KT) \le \omega_i$).* (ii) *If occupancies are constant in time, each tenant's time-integrated dominant share equals its instantaneous dominant share $\max_r k_i^r/C^r$, so the entitlement cap coincides with snapshot dominant-share fairness (classical DRF [24]); the two diverge precisely when occupancy varies over time.*

Proof. (i) With $m = 1$ the dominant share is the single share $s_i = A_i/(KT)$ and the cap $s_i \le \omega_i$ is the scalar memory-share entitlement of section 3.1. *(ii)* If each $\mathbf{k}_i(t) \equiv \mathbf{k}_i$ is constant, then $\mathbf{M}_i(T) = \mathbf{k}_i T$ and $s_i^r = M_i^r/(C^r T) = k_i^r/C^r$ is time-independent; the dominant share is then the static $\max_r k_i^r/C^r$ that classical DRF equalizes. For time-varying occupancy the time-integral and the snapshot differ, so the coincidence is special to the constant case. □

The idealized allocation-rule counterpart—leximin time-integrated DRF, and the sense in which it specializes to classical DRF as a *rule*—is developed in section C; it is not used below.

Proposition 8 says the capacity meters *agree among themselves* on a single capacity axis. The same collapse has a sharp negative consequence for a *flow* axis, which we record now and measure in section 5.3: every occupancy meter, however time-aware, is blind to transfer.

Proposition 9 (Capacity meters are irreducibly blind to flow)**.** *On a single capacity (stock) axis r, the time-aware dominant-share meter (SDRF on occupancy) reduces identically to the time-integrated occupancy share: with one resource, $\max_s M_i^s/(C^s\,\Delta)$ has the lone term $M_i^r/(C^r\Delta)$, the same scalar a naive occupancy meter reports. Hence no occupancy meter on that axis—naive or time-integrated—contains any function of the transfer flow $\int \tau_i^r\ \mathrm{d}t$; a tenant's stock and its flow are not determined by one another, so a stock meter cannot bound flow. Only a meter on the bandwidth axis $\int \tau_i^r\ \mathrm{d}t$ (definition 4) sees flow at all.*

Proof. The first claim is the $m = 1$ collapse of the dominant share, exactly as in proposition 8(i) applied to occupancy. For the second, occupancy $k_i^r(t)$ (blocks resident) and transfer $\tau_i^r(t)$ (bytes/s crossing the link) are independent coordinates of the residency vector: a tenant can hold few resident blocks while moving many—spilling and reloading—so $\int k_i^r$ and $\int \tau_i^r$ are not functions of one another, and any meter that is a function of $\{k_i^s\}$ alone is constant along variation of τ_i^r. The bandwidth axis is the only coordinate carrying the flow term. □

The isolation theorem bounds usage from *above*: no tenant exceeds its entitlement. Fair allocation also needs the opposite direction—a tenant that wants service actually *gets* roughly its share, and is not starved. We prove that too. The key is to reason about *occupancy-time over an interval* (the integral), not occupancy at a single instant, because a competitor can momentarily exceed its share by spending burst credit; only its integral is bounded.

Theorem 3 (Backlog service: no starvation)**.** *Fix a scarce axis r and an interval $[t_1, t_2]$ on which tenant i is* fluidly backlogged at its entitled rate and bucket-feasible*: at every instant its pending eligible work that fits on r is enough to occupy at least its entitled share $\omega_i C^r$, and its bucket on r stays strictly above its overdraft floor $-H_i^r$ (so i is always eligible for placement). The phrase "enough to occupy its entitled share" is what distinguishes this from merely having one small runnable request: i must have the* volume *of work to fill its share whenever room appears. Assume:*

- *(i) the placement policy is* work-conserving*—it never leaves usable capacity on r idle while some eligible runnable request fits (the* GreedyKV *rule of section 4.1 is work-conserving by construction, so this holds for the mechanism we deploy);*
- *(ii) every other tenant j is regulated by its bucket, so by theorem 2 its occupancy-time over the interval satisfies $M_j^r(t_2) - M_j^r(t_1) \le \omega_j C^r (t_2 - t_1) + B_j^{r,\max} + H_j^r$;*
- *(iii) entitlements are admissible, $\sum_j \omega_j \le \eta < 1$;*

(iv) fragmentation leaves at least an η fraction of r usable at all times; and

(v) (aggregate saturation) *the tenants'* combined *eligible bucket-feasible demand is enough to occupy the usable ηC^r capacity at all times, except during switching/granularity transients of total duration $\tau_{\text{idle}}(t_1, t_2)$.*

Then

$$M_i^r(t_2) - M_i^r(t_1) \;\geq\; \omega_i C^r (t_2 - t_1) \;-\; \underbrace{\Big(\textstyle\sum_{j\neq i} \big(B_j^{r,\max} + H_j^r\big) + C^r\, \tau_{\text{idle}}(t_1, t_2)\Big)}_{\textit{aggregate slack}}.$$

If moreover the transient is asymptotically negligible, $\tau_{\text{idle}}(0, T) = o(T)$, then $\liminf_{T\to\infty} M_i^r(T)/T \geq \omega_i C^r$: in the long run tenant i receives at least its entitled rate.

Remark 5 (Why two demand conditions, and what each does)**.** *There are deliberately two demand assumptions, and they play different roles. The* tenant-level *fluid backlog (in the preamble) guarantees that i personally has enough work to absorb its share whenever space is offered to it. The* aggregate *saturation (v) guarantees that the axis as a whole stays busy—so the capacity i is entitled to is actually being produced and not sitting empty. Neither implies the other: i could have ample work while the system overall is under-loaded (then the axis idles and nobody is starved, so the guarantee is moot), or the system could be saturated while i has too little work to claim its share. Assumption (v) is the natural "the resource is contended" regime; under-loaded systems do not need a service guarantee at all.*

Proof. The proof is one subtraction: (total occupancy-time on the axis) minus (what everyone else could have taken) is what is left for i. We bound the two pieces.

Step 1: the axis is almost fully used. By assumptions (iv) and (v), at least ηC^r of capacity is usable at every instant, and the tenants' combined eligible demand suffices to occupy it except during the transient τ_{idle}; by work-conservation (i) the policy actually places that work rather than idling. Therefore the total occupancy-time consumed on r, summed over all tenants, is at least the usable capacity-time minus the idle time:

$$\sum_j \big(M_j^r(t_2) - M_j^r(t_1)\big) \;\geq\; \eta C^r (t_2 - t_1) - C^r\, \tau_{\text{idle}}(t_1, t_2).$$

(Tenant i's own fluid backlog is what lets i *claim* a share of this busy capacity rather than being skipped for lack of work; aggregate saturation is what makes the capacity busy in the first place. Step 3 turns "the axis is busy and i can claim its part" into a concrete lower bound on i.)

Step 2: everyone except i is bounded by their buckets. Summing assumption (ii) over $j \neq i$ and using $\sum_{j\neq i} \omega_j \leq \eta - \omega_i$ from (iii),

$$\sum_{j\neq i} \big(M_j^r(t_2) - M_j^r(t_1)\big) \;\leq\; (\eta - \omega_i) C^r (t_2 - t_1) + \sum_{j\neq i} \big(B_j^{r,\max} + H_j^r\big).$$

Step 3: subtract. The total over all tenants equals i's share plus the others' shares, so i's share is the total (Step 1) minus the others' (Step 2). The only subtlety is that the bare subtraction gives a lower bound on i's *available* room; i's fluid backlog (the preamble) ensures i has the work to actually occupy it, so the bound is achieved:

$$\begin{aligned} M_i^r(t_2) - M_i^r(t_1) &\geq \Big[\eta C^r (t_2 - t_1) - C^r \tau_{\text{idle}}\Big] - \Big[(\eta - \omega_i) C^r (t_2 - t_1) + \textstyle\sum_{j\neq i} (B_j^{r,\max} + H_j^r)\Big] \\ &= \omega_i C^r (t_2 - t_1) - C^r \tau_{\text{idle}} - \textstyle\sum_{j\neq i} (B_j^{r,\max} + H_j^r). \end{aligned}$$

The η terms cancel, leaving exactly the entitled rate $\omega_i C^r$ minus the aggregate slack. Dividing by $T = t_2 - t_1$ with $t_1 = 0$: the bucket terms are a fixed constant, so divided by T they vanish; the idle term contributes $C^r\, \tau_{\text{idle}}(0, T)/T$, which tends to 0 precisely under the stated condition $\tau_{\text{idle}}(0, T) = o(T)$. Hence $\liminf_T M_i^r(T)/T \geq \omega_i C^r$. □

Remark 6 (The hypotheses, inventoried)**.** *The service bound is deliberately assumption-explicit, and each hypothesis is either discharged by construction or assigned a measurement.* Work conservation *(i) holds for the deployed rule by construction (section 4.1); it is a property of the selector, not of the workload.* Bucket

regulation of competitors *(ii) is theorem 2, enforced by the mechanism itself.* Admissible entitlements *(iii) and the* usable-capacity floor *(iv) are operator-controlled configuration, with η absorbing fragmentation; their stress-test under fragmentation spikes and expensive preemption we leave to future work.* Aggregate saturation *(v) is the one genuinely workload-dependent hypothesis—it states that the bound speaks to busy regimes, which is where starvation is at stake; off-saturation the bound is slack but the tenant is also not starving. The realized ratio of under-service to aggregate slack is measured directly in section 5 (five of five seeds at every bucket depth), so departures from any hypothesis would surface as violations there; none do in the regimes tested.*

Remark 7 (Isolation + service = fair allocation)**.** *Theorem 2 (at most $\omega_i C^r$ up to i*'s own *burst slack) and theorem 3 (at least $\omega_i C^r$ up to the* aggregate *burst slack of others) together pin a backlogged conformant tenant's long-run rate on the scarce axis to its entitlement $\omega_i C^r$. This two-sided guarantee is the memory-time analogue of the service-curve sandwich in weighted fair queuing, and it is what makes "entitlement" a promise rather than only a cap. The asymmetry in the slack terms is expected and correct: a tenant's own burst lets it briefly overshoot (isolation slack $B_i^{r,\max} + H_i^r$), while others' bursts are what can briefly delay it (service slack $\sum_{j\neq i}(B_j^{r,\max} + H_j^r)$).*

3.4 Priority and SLO tiers as entitlement structure

Having fixed what a fair allocation of residency is, we show that priority and SLO tiers fit the same object as a separate knob, without being able to steal share. Production platforms expose priority classes—"standard," "critical," "batch"—and per-class SLOs on time-to-first-token (TTFT) and time-between-tokens (TBT). A tempting shortcut is to encode a higher tier as a larger memory-time entitlement ω_i. That is incorrect, and the reason is instructive: it conflates two orthogonal quantities. We show that priority decomposes cleanly into three knobs, all already present in the framework, and that this decomposition yields a safety property—urgency cannot be used to steal share.

3.5 Share is not urgency

Consider a light-but-critical tenant (rare requests, tight TTFT) and a heavy-but-elastic tenant (sustained load, delay-tolerant). The critical tenant needs to win the resident-set competition *when it is present*, but consumes little long-run memory-time; the elastic tenant needs a large long-run *share* but tolerates delay. Encoding criticality as a large ω serves neither: it grants the light tenant an entitlement it never uses, yet—because ω is a long-run average rate—still does not guarantee it fast service at its moment of arrival. Long-run share and instantaneous urgency are independent design dimensions.

Definition 8 (Priority tier)**.** *A tenant's service class is a triple $(\omega_i,\ \beta_i,\ u_i)$:*

1. *the* share *ω_i, its long-run dominant memory-time entitlement (rate), as in definition 7;*
2. *the* burst profile *$\beta_i = (B_i^{r,\max}, H_i^r)_r$, the per-axis bucket depth and overdraft, governing how large and sustained a surge the tenant may take before throttling (theorem 2);*
3. *the* urgency weight *$u_i \geq 0$, which scales the tenant's progress density in the placement objective.*

A higher SLO tier is a coordinate-wise increase in (ω_i, β_i, u_i), not a scalar bump in ω_i.

Urgency enters the master objective eq. (2) only through the progress term: a request's contribution becomes $u_{\tau(r)}\,\Psi(s_{\tau(r)}(t))\,\rho_r(t)$, optionally with a deadline-aware ρ_r that rises as a TTFT/TBT deadline approaches. Crucially, u_i multiplies the *ordering* term but does not touch the bucket dynamics.

3.6 Isolation is urgency-invariant

The central safety property is that the long-run fairness guarantee is independent of urgency: turning a tenant's urgency up improves its when-present scheduling rank but cannot raise its long-run share above entitlement.

Theorem 4 (Urgency-invariant isolation)**.** *Fix any profile of urgency weights $\{u_i\}$ and any nondecreasing Ψ. If each tenant i is regulated by the per-axis buckets of theorem 2 with rate $\omega_i C^r$, depth $B_i^{r,\max}$, and overdraft H_i^r, then for every axis r and interval $[t_1, t_2]$,*

$$M_i^r(t_2) - M_i^r(t_1) \le \omega_i C^r (t_2 - t_1) + B_i^{r,\max} + H_i^r,$$

and hence $\limsup_{T\to\infty} s_i(T) \le \omega_i$, regardless of $\{u_i\}$.

Proof. The bound of theorem 2 is a property of the bucket regulator alone: it follows from the refill rate $\omega_i C^r$, the saturation cap $B_i^{r,\max}$, and the overdraft floor $-H_i^r$, via the reflected-accumulator identity. The placement objective eq. (2)—including the factor $u_{\tau(r)}\Psi(\cdot)\rho_r$—only selects *which* bucket-feasible set is resident at each instant; it cannot cause consumption while a bucket is at its floor, because enforcement forbids it. Thus the accumulator identity and the three inequalities used in the proof of theorem 2 are unchanged for any $\{u_i\}$ and any monotone Ψ, and the bound follows verbatim. Taking $t_1 = 0$, dividing by $C^r T$, and maximizing over the finite axis set gives the dominant-share bound. □

Remark 8 (Separation of concerns)**.** *Theorem 4 is the formal statement that* urgency governs ordering and entitlement governs rate*. The two knobs are non-interfering: a platform can hand a tenant a high urgency weight to meet a tight latency SLO without any risk that the tenant thereby captures more than its configured long-run share. This is what makes priority classes safe to expose.*

3.7 Self-declared urgency is share-neutral

The decomposition also makes self-declared priority safe. If urgency is platform-assigned (contracted per service class), it cannot affect long-run share. If it is self-declared—a tenant tags its own requests "critical"—the bucket still caps share, so over-declaration is bounded in its effect.

Proposition 10 (Share-neutrality of self-declared urgency)**.** *Suppose tenant i may freely choose the urgency weights it attaches to its own requests, with all other tenants fixed. Then no choice of urgency weights increases tenant i's long-run dominant memory-time share beyond ω_i. The only effect of raising urgency is to reorder i's own requests relative to one another and relative to others' requests when i's bucket is not the binding constraint.*

Proof. The long-run share bound of theorem 4 holds for every urgency profile, in particular for every unilateral choice by i; hence $\limsup_T s_i(T) \le \omega_i$ irrespective of i's declarations. Within that cap, raising u_i increases the objective contribution of i's requests, which can change the selected resident set only by promoting i's requests in the ordering; once i's bucket reaches its floor, enforcement excludes further consumption regardless of u_i. Thus the sole degree of freedom urgency confers is intertemporal and intra-tenant reordering, not additional share. □

Remark 9 (Why this matters for deployment)**.** *Proposition 10 means a platform can offer self-service priority tagging without letting it shift long-run share: a tenant that marks everything critical merely spends its own bucket faster on its own most-promoted requests and is throttled at the same long-run rate. Raising the declared urgency of work therefore leaves long-run share unchanged (proposition 10, share-neutrality); its only effect is to reorder a tenant's own requests within its bucket, while the long-run fair allocation is untouched.*

3.8 Hard deadlines as feasibility, not fairness

Soft urgency is an objective weight; a hard TTFT/TBT deadline is a feasibility constraint. A request with deadline d_r requires a placement that keeps it sufficiently resident to complete by d_r; this restricts the admissible-placement set $\Phi_r(t)$ and hence the feasibility family $\mathcal{F}(t)$, exactly as fragmentation and tier-residency constraints do (definition 3). The urgency-weighted objective then schedules within the deadline-feasible region, recovering an earliest-deadline-first flavor when ρ_r is taken to grow as d_r approaches. We deliberately do not claim a real-time schedulability guarantee: whether a given deadline mix is jointly feasible is a separate admission-control question that composes with, but is not implied by, the meter. The contribution here is that deadlines and priorities slot into the same placement problem as everything else, without a new primitive.

4 Placement as Unified Control

Pillar 4: how to compute the allocation online. Admission, continuation, preemption, routing, tiering, and prefill/decode disaggregation are not separate mechanisms—they are one online placement problem over residency, solved by a single urgency-weighted rule. We fix intuition on the single-axis scheduler, state the master reduction that unifies the operations, give the one selection rule and its guarantees—$\frac{1}{m+1}$ unconditionally on m axes, $\frac{1}{2}$ in the single-binding-axis regime—and record the cluster/instance two-plane decomposition with routing as the cluster case.

We first develop the single-axis ($m = 1$) scheduler, which is the simplest instance of the general placement problem and fixes intuition; section 4.3 gives the full vector/urgency-aware form.

At time t, let $\mathcal{R}(t)$ be the set of runnable candidate requests: waiting requests, currently resident requests, and resumable preempted requests. Let $S^-(t)$ be the previously resident set. A set $S \subseteq \mathcal{R}(t)$ is feasible if it can be packed into KV memory; write $S \in \mathcal{F}(t)$. In the fluid model,

$$S \in \mathcal{F}(t) \quad \Longleftrightarrow \quad \sum_{r \in S} k_r(t) \le K.$$

With block fragmentation, prefix locality, or paging constraints, $\mathcal{F}(t)$ is a more general packing-feasibility family.

The scheduler repeatedly solves

$$S^\star(t) \in \arg\max_{S \in \mathcal{F}(t)} \left\{ \sum_{r \in S} \Phi\big(B_{\tau(r)}(t)\big)\, \rho_r(t) - \sum_{r \in S^-(t) \setminus S} E_r(t) - \sum_{r \in S \setminus S^-(t)} R_r(t) \right\}. \tag{1}$$

Here $\tau(r)$ is the tenant owning request r, Φ is increasing, $\rho_r(t)$ is a progress/SLO density, $E_r(t)$ is eviction cost, and $R_r(t)$ is resume or recomputation cost. Prefix caching affects E_r and R_r: evicting a request with cheap reusable prefix has lower cost than evicting one whose KV must be recomputed.

Remark 10 (Admission, continuation, and preemption are not separate primitives)**.** *A request is admitted when it enters $S^\star(t)$. It continues when it remains in $S^\star(t)$. It is preempted when it leaves $S^\star(t)$. Thus scheduling and preemption solve the same optimization problem at different times.*

4.1 Tractability of the resident-set problem

Equation (1) is an optimization over feasible packings, and in general it is hard. We make this precise and then give a usable approximation.

Proposition 11 (Hardness)**.** *Even with zero switching and resume costs ($E_r = R_r = 0$) and the fluid feasibility family $\mathcal{F}(t) = \{S : \sum_{r \in S} k_r(t) \le K\}$, maximizing eq. (1) is NP-hard.*

Proof. With $E_r = R_r = 0$ the objective is $\sum_{r \in S} w_r$ with $w_r = \Phi(B_{\tau(r)}(t))\rho_r(t) \ge 0$, subject to $\sum_{r \in S} k_r(t) \le K$. This is exactly the 0/1 knapsack problem with item values w_r, item sizes $k_r(t)$, and capacity K, which is NP-hard. Switching/resume costs only generalize the instance. □

Hardness is unsurprising; the practical question is whether a fast, deployable rule has a guarantee. Define the *cost-effectiveness density* of a candidate request as

$$\theta_r(t) = \frac{\Phi(B_{\tau(r)}(t))\, \rho_r(t) - R_r(t)\, \mathbf{1}[r \notin S^-(t)]}{k_r(t)},$$

i.e. adjusted progress value per resident KV token, net of the resume cost paid if r must be (re)admitted. The greedy selector GreedyKV sorts candidates by $\theta_r(t)$ in decreasing order and admits them while KV capacity remains, charging eviction cost E_r for any previously resident request displaced.

The rule is *work-conserving* by construction: it fills usable capacity with any eligible runnable request that fits rather than leaving it idle, and it evicts an incumbent only to seat a strictly higher-scoring request that needs the freed space—never to idle. In particular an over-entitlement tenant whose balance has reached

the overdraft floor is barred from *new* consumption (it is no longer eligible), but an incumbent that is merely overdrawn is not evicted while its capacity would otherwise sit unused. This makes "overdrawn" an admission gate, not a forced eviction, and it is what discharges the work-conservation hypothesis of theorem 3; the score θ_r governs only the *order* in which capacity is filled and which incumbent yields under genuine contention, never whether to leave capacity empty.

Theorem 5 (Greedy guarantee)**.** *Suppose progress values and resume costs are such that the resulting item values $\hat{w}_r = \Phi(B_{\tau(r)}(t))\rho_r(t) - R_r(t)\mathbf{1}[r \notin S^-(t)]$ are nonnegative, and suppose no single request exceeds capacity, $k_r(t) \le K$. Then on the fluid feasibility family the better of* GreedyKV *and the single best feasible request achieves at least $\frac{1}{2}$ of the optimal objective of eq. (1) ignoring eviction costs, and at least $\frac{1}{2}\,\mathrm{OPT} - \sum_{r\in S^-(t)} E_r(t)$ once eviction costs are charged.*

Proof. Ignoring eviction costs the problem is a one-dimensional knapsack: each candidate request is an item of value $\hat{w}_r \ge 0$ and size $k_r(t) \le K$, and we want the highest-value subset that fits in capacity K. Order items by density $\hat{w}_r/k_r$ and add them in that order until the next one, call it $j^\star$, no longer fits. Write G for the total value of the items added before $j^\star$. The fractional knapsack (which allows taking a fraction of $j^\star$) has value $G + (\text{fraction}) \cdot \hat{w}_{j^\star} \le G + \hat{w}_{j^\star}$, and the fractional optimum is at least the integer optimum OPT; hence $G + \hat{w}_{j^\star} \ge \mathrm{OPT}$. Also $\hat{w}_{j^\star} \le \mathrm{OPT}$, because the single item $\{j^\star\}$ is itself a feasible subset. Adding these two facts, at least one of G and $\hat{w}_{j^\star}$ is $\ge \frac{1}{2}\mathrm{OPT}$. GreedyKV returns the better of its greedy prefix (value $\ge G$) and the single best feasible item (value $\ge \hat{w}_{j^\star}$), so it attains $\ge \frac{1}{2}\mathrm{OPT}$. Charging eviction subtracts at most $\sum_{r\in S^-(t)} E_r(t)$ from the realized objective, giving the second bound. □

Remark 11 (Why this is enough in practice)**.** *The eviction-cost slack $\sum_{r\in S^-(t)} E_r(t)$ is paid only for requests actually displaced in an epoch, and prefix caching makes E_r small for requests whose KV is cheaply reconstructible. In the common regime where the resident set changes slowly between epochs, the realized objective tracks $\frac{1}{2}\,\mathrm{OPT}$ closely. The deployed rule's empirical study—a control-plane ablation comparing it against residency-blind alternatives using only observability a production engine has, with no per-epoch oracle—we leave to future work.*

4.2 Placement is everything: one online problem

Theorem 6 (Master reduction)**.** *Admission, continuation, preemption, multi-instance routing, KV tiering, and PD disaggregation are special cases of a single online optimization: at each epoch choose a feasible placement field $\boldsymbol{\varphi}(t) \in \mathcal{F}(t)$ maximizing entitlement-weighted progress net of switching cost,*

$$\boldsymbol{\varphi}^\star(t) \in \arg\max_{\boldsymbol{\varphi}\in\mathcal{F}(t)} \Big\{ \textstyle\sum_{r\ live} \Psi\big(s_{\tau(r)}(t)\big)\, \rho_r(t) - \mathrm{Switch}\big(\boldsymbol{\varphi}, \boldsymbol{\varphi}(t^-)\big) \Big\}, \tag{2}$$

where Ψ is decreasing in the owner's dominant memory-time share, ρ_r is progress density, and Switch *aggregates eviction, resume, recompute, and cache-move costs.*

Proof. We exhibit each operation as a restriction of the admissible sets Φ_r and axis set. *Admission/continuation/preemption*: with $m = 1$ and $\Phi_r \in \{\mathbf{0},\, k_r e_1\}$ (out or resident), eq. (2) is the resident-set selection eq. (1); entering the support is admission, staying is continuation, leaving is preemption (section 4.1). *Routing*: let axes be indexed by instance g and require each request to place on exactly one instance's axes, $\Phi_r = \{\,\text{occupancy on some single } g\,\}$; choosing the supporting g is routing, and heterogeneous (π_g, δ_g, C_g) are heterogeneous axis rates/capacities. *Tiering*: let axes include HBM, DRAM, NVMe with a per-request admissible set that allows splitting its KV across tiers subject to a residency constraint; choosing the split is the tiering policy, and Switch charges promotion/demotion bandwidth. *PD disaggregation*: separate prefill and decode axes (possibly on separate pools), with the admissible set encoding that decode cannot run before prefill has produced KV; the KV-transfer axis carries the handoff. In every case the objective and feasibility are instances of eq. (2), differing only in $\{\Phi_r\}$ and the axis set. □

Theorem 6 is the structural heart of the paper: the operations that the literature treats as separate subsystems are coordinates of one feasible-placement problem, and the meter is the time-integral of its solution.

Remark 12 (Concrete instantiation of the objective terms)**.** *Equation* (2) *is a template; the three free terms have canonical choices we use throughout and in the experiments, and the guarantees do not depend on their fine tuning. The* share-penalty Ψ *is any decreasing function of the owner's dominant memory-time share—we use* $\Psi(s) = \Phi(B)$*, the bucket balance* $B_{\tau(r)}(t)$ *itself (positive balance* $\Rightarrow$ *high priority, drained balance* $\Rightarrow$ *excluded), which is the link between the objective and the leaky-bucket regulator and is why the fairness theorems transfer to the deployed rule. The* progress density $\rho_r(t)$ *is the request's expected useful progress per unit time at the candidate placement—tokens/s of forward progress, optionally weighted by a soft urgency factor* $u_{\tau(r)}$ *for SLO tiers (section 3.4); the simplest instantiation* $\rho_r \equiv 1$ *already yields a work-conserving residency-fair scheduler, and urgency only reorders within the bucket-feasible set (remark 13). The* switching term Switch *aggregates measured eviction, resume/recompute, and cache-move bytes; prefix locality enters here, since a request whose prefix is already resident has near-zero resume cost. A* hard *deadline is not an objective term at all but a feasibility constraint on* Φ_r *(section 3.8). None of these choices is load-bearing for the isolation and service guarantees, which rest only on the bucket; they govern* efficiency *(which feasible set is chosen), not* fairness *(which sets are feasible), and the empirical study fixes them to the canonical values above.*

4.3 One rule for scheduling, routing, preemption, and eviction

Theorem 6 reduces every operation to one online placement problem, and section 4.1 showed the per-epoch problem is NP-hard with a greedy constant-factor approximation. This subsection makes explicit how the four operations the systems literature treats separately—admission/scheduling, routing, preemption, and KV eviction—fall out of one selection rule once urgency (section 3.4) and the resource vector (section 2.2) are included.

4.4 The urgency-weighted density rule

At each epoch the scheduler scores every candidate placement of every live request by its *urgency-weighted cost-effectiveness density*

$$\theta_r(t) = \frac{u_{\tau(r)}\,\Psi\big(s_{\tau(r)}(t)\big)\,\rho_r(t) - R_r(t)\,\mathbf{1}[r \notin S^-(t)]}{\mathbf{c}^\top \boldsymbol{\varphi}_r(t)},$$

the entitlement- and urgency-weighted progress per unit weighted memory-time, net of resume cost. The unified selector PlaceKV considers candidate placements in decreasing θ_r and commits each if its axes admit it under definition 3, charging eviction cost for any displaced incumbent. Ties in θ_r are broken toward the candidate with the smaller binding-axis footprint (and, if still tied, the lower tenant balance, favoring the tenant further below entitlement); the choice affects only the order of equal-density items and not the approximation ratio, since the analysis bounds the greedy prefix value regardless of tie resolution. A candidate that does not fit on the binding axis is simply skipped—placements are atomic, with no partial admission—and reconsidered only if eviction of an over-budget incumbent frees room within the same epoch.

Remark 13 (Urgency cannot punch through the cap)**.** *The selector only ever commits* bucket-feasible *placements: a tenant whose bucket on the relevant axis is at its overdraft floor* $-H_i^r$ *is excluded from consideration on that axis regardless of its urgency weight* u_i *or progress density* ρ_r*. Urgency and switching cost reorder candidates* within *the bucket-feasible set; they never enlarge it. This is exactly why theorem 4 holds: the objective in eq.* (2) *decides which feasible set is resident, while the bucket decides what is feasible, and no objective term can override a drained bucket.*

The four operations are read off the resulting placement field:

1. **Scheduling/admission**: a waiting request whose best placement is committed enters the resident set.
2. **Routing**: among per-instance candidate placements (theorem 6), the committed one selects the instance; cache-move cost enters R_r.
3. **Preemption**: an incumbent whose density is dominated by admitted competitors is dropped from the field.

4. **Eviction**: when an axis is full, the displaced KV is exactly the lowest-θ_r resident mass on that axis, optionally demoted to a cheaper tier (a tiering placement) rather than discarded.

Remark 14 (Eviction is dual to admission: a design principle)**.** PLACEKV *unifies eviction with admission through a single ordering. In the scalar single-binding-axis case this is exact: the algorithm commits placements in decreasing density $\theta_{r^\star}$ until the axis saturates, so the marginal incumbent displaced to admit a higher-density candidate is precisely the lowest-$\theta_{r^\star}$ resident mass on that axis—the standard greedy-knapsack duality between including a new item and excluding the marginal one. Restricting displacement to* over-budget *incumbents—those whose owner has nonpositive bucket balance on the binding axis, i.e. is already at or above its entitled occupancy—ensures bucket-positive conformant tenants are never evicted to serve over-budget ones (consistent with theorem 4).* Tiered demotion *is the variant in which the evicted mass is re-placed on a slacker axis at its cache-move cost rather than discarded, a tiering placement in Φ_r. In genuinely multi-axis epochs the precise "complementary set" statement no longer holds verbatim (it inherits the multidimensional-knapsack subtleties of theorem 7); we use it as a design principle and measure eviction quality empirically.*

4.5 Guarantee and overhead

Theorem 7 (Greedy guarantee under a single binding axis)**.** *Suppose the urgency-weighted item values $\hat{w}_r = u_{\tau(r)}\Psi(s_{\tau(r)})\rho_r - R_r\mathbf{1}[r \notin S^-]$ are nonnegative, no single request exceeds any axis capacity, and the epoch has a* single binding axis *$r^\star$: on every other axis the total demand of all candidate requests fits within capacity, so only $r^\star$ can constrain the selection. Then the better of* PLACEKV *and the single best feasible request attains at least $\frac{1}{2}$ of the optimum of eq. (2) ignoring eviction costs, and at least $\frac{1}{2}\,\mathrm{OPT} - \sum_{r\in S^-(t)} E_r(t)$ once eviction is charged.*

Proof. The argument is the standard greedy-knapsack bound, made concrete. Because every axis other than $r^\star$ can hold all candidates simultaneously, those axes never rule out a selection; a set of requests is feasible if and only if its total occupancy on $r^\star$ fits in $C^{r^\star}$. The problem is therefore a one-dimensional knapsack *on the binding axis*: items are the candidate requests, item r has value $\hat{w}_r \geq 0$ and *size* $\varphi_r^{r^\star}$ (its occupancy on the binding axis), and the capacity is $C^{r^\star}$. The matching density is value-per-binding-size,

$$\theta_r^{r^\star} = \frac{\hat{w}_r}{\varphi_r^{r^\star}},$$

and we run the greedy in this order. Let j be the first item the greedy order cannot fit. Two facts are classical. First, the fractional relaxation (which allows a fraction of j and upper-bounds OPT) equals the greedy prefix value plus a fraction of $\hat{w}_j$, so the greedy prefix value G satisfies $G + \hat{w}_j \geq \mathrm{OPT}$. Second, $\hat{w}_j \leq \mathrm{OPT}$, since $\{j\}$ is itself a feasible set. Adding, at least one of G and $\hat{w}_j$ is $\geq \frac{1}{2}\mathrm{OPT}$; PLACEKV returns the better of its greedy prefix and the single best feasible request, hence $\geq \frac{1}{2}\mathrm{OPT}$. The urgency weight $u_{\tau(r)}$ only multiplies values and leaves the ratio unchanged. Charging eviction removes at most $\sum_{r\in S^-(t)} E_r(t)$. □

Remark 15 (Scoring in practice)**.** *The implementation in section 4.3 scores candidates by the weighted density $\hat{w}_r/(\mathbf{c}^\top\boldsymbol{\varphi}_r)$, which is convenient because it needs no advance knowledge of which axis will bind. The formal $\frac{1}{2}$ guarantee of theorem 7 uses the binding-axis density $\hat{w}_r/\varphi_r^{r^\star}$; the two orderings coincide, and the guarantee transfers exactly, when the weighted cost $\mathbf{c}^\top\boldsymbol{\varphi}_r$ is proportional to the binding-axis occupancy $\varphi_r^{r^\star}$ (for instance when the binding axis dominates the weighted cost). Outside that case the weighted-density score is a heuristic, evaluated empirically in section 5.*

4.6 The general multi-axis guarantee: reduction to m-dimensional knapsack

When two or more axes can bind, the per-epoch problem is a multidimensional knapsack: still NP-hard, and now without an efficient approximation scheme. The same density-ordered greedy, augmented with the single best feasible candidate, nevertheless retains an *unconditional* constant-factor guarantee whose constant degrades only with the number of axes m—which in our setting is small and fixed (two for prefill/decode disaggregation, a few more with tiers).

Theorem 8 (Multi-axis greedy guarantee). *Suppose $\hat{w}_r \geq 0$ and no single candidate exceeds any axis capacity. With m residency axes, the better of* PlaceKV*'s greedy set and the single best feasible candidate attains at least $\frac{1}{m+1}\,\mathrm{OPT}$ of the eviction-free objective of eq. (2), and at least $\frac{1}{m+1}\,\mathrm{OPT} - \sum_{r\in S^-(t)} E_r(t)$ once eviction is charged.*

Proof. The argument is the reduction of theorem 5 carried to m dimensions; nothing is dropped or projected. The per-epoch problem is, verbatim, an m-dimensional 0/1 knapsack: items are the candidate placements, item r has value $\hat{w}_r \geq 0$ and the m-vector of sizes $\boldsymbol{\varphi}_r$, and the m capacities are $(C^s)_{s=1}^m$. For m-dimensional 0/1 knapsack, density-ordered greedy augmented with the single best feasible item is a $\frac{1}{m+1}$-approximation [39, 40]: the LP relaxation upper-bounds OPT and at a basic optimum at most m variables are fractional (one per capacity row), so the integral part of the LP optimum is a feasible set whose deficit against OPT is at most the m fractional items' value; each fractional item is singly feasible, so its value is at most that of the best single feasible candidate, and the better of the greedy set and that candidate therefore attains $\frac{1}{m+1}\,\mathrm{OPT}$. Urgency weights scale values without changing the ratio; charging eviction subtracts at most $\sum_{r\in S^-} E_r$. □

Corollary 2 (Single-axis recovery). *For $m = 1$ the bound is $\frac{1}{2}\,\mathrm{OPT}$, recovering theorem 5.*

Remark 16 (Two complementary bounds). *Theorem 7 and theorem 8 are complementary, not competing: the former gives the sharper $\frac{1}{2}$ under the stronger hypothesis that only one axis can constrain the selection; the latter gives $\frac{1}{m+1}$ with no hypothesis on which axes bind. Together they bracket the deployed rule: $\frac{1}{m+1}$ always, $\frac{1}{2}$ in the single-binding-axis regime.*

Remark 17 (Why we stop at the simple bound). *We do not pursue a sharper approximation* scheme*: multidimensional knapsack admits no EPTAS even at $m = 2$ unless $\mathrm{P} = \mathrm{NP}$ [41]. The $\frac{1}{m+1}$ reduction bound is the appropriate worst-case guarantee for a fast deployable per-epoch rule, and because m is a small fixed constant the bound is concrete—$\frac{1}{3}$ for two-axis disaggregation—rather than a vacuous large-dimension statement.*

Remark 18 (Worst-case floor versus expected operating point). *Theorem 8 is a conservative floor over all instances on m axes. What governs the difficulty of a* given *epoch is how many axes simultaneously constrain the selection, and in prefill/decode–disaggregated serving we expect this count to concentrate at one: a request's dominant axis migrates from prefill to decode over its lifetime, so an epoch is typically prefill-bound* or *decode-bound, not both, placing the system in the single-binding-axis regime where the sharper $\frac{1}{2}$ of theorem 7 applies. The binding-count distribution is directly observable from the deployed rule's own admission trace—no oracle is required—and its measurement, together with a control-plane ablation that evaluates each placement decision against a residency-blind alternative, is left to future work. We claim the floor as theory and scope the operating-point measurement as experiment, and are explicit that the $\frac{1}{2}$-regime claim is a prediction the measurement will test.*

Remark 19 (Complexity). *Scoring is linear in the number of candidate placements; sorting dominates at $O(n \log n)$ per epoch for n candidates, with incremental updates between epochs when the candidate set changes slowly. The rule's observability needs are exactly the per-tenant, per-axis occupancy and bucket state the meter already maintains (section 3.1); no auxiliary solver or oracle is required at any cadence. Its empirical evaluation—the control-plane ablation—is left to future work.*

Remark 20 (Cadence and anti-thrashing). PlaceKV *runs at epoch cadence—one pass per scheduling epoch, between batch iterations—so its decisions amortize over the iteration the engine was going to take anyway; nothing in the rule requires sub-iteration reaction. Two sources of oscillation are damped by construction and one by a standard guard. Bucket balances drain continuously, so a tenant does not flap across the eligibility threshold within an epoch; and eviction is restricted to* over-budget *incumbents displaced by strictly higher-density candidates, so a freshly admitted request cannot be evicted by the marginal candidate it just displaced. The remaining risk—a tenant whose balance hovers at the overdraft floor being admitted and barred on alternating epochs—is removed by hysteresis on the eligibility threshold (re-eligibility at a balance strictly above the floor), a one-parameter guard whose width trades reaction latency against churn and whose cost is bounded by the same switching terms the objective already charges.*

Algorithm 1 PLACEKV: unified placement per epoch

1: Update per-axis usage $\mathbf{M}_i$ and bucket balances $\mathbf{B}_i$ (refill $\omega_i \mathbf{C}$, drain measured occupancy).
2: Form candidate placements: for each live/waiting/resumable request, its admissible placements $\Phi_r(t)$ across tiers and instances.
3: Score each candidate by urgency-weighted density $\theta_r(t)$.
4: **for** candidates in decreasing θ_r **do** commit if axes admit (definition 3); else, if displacing a strictly lower-θ *over-budget* incumbent frees the axis, demote that incumbent to a slacker tier or evict it, then commit. ▷ An incumbent is *over-budget* on axis r if its owner's bucket balance there is nonpositive, $B_j^r(t) \leq 0$; equivalently its owner is already at or above its entitled occupancy on r. A *bucket-positive* (conformant, below-entitlement) incumbent is never displaced—this is what theorem 4 relies on.
5: Realize the field: admit new commits, continue retained, preempt/evict dropped, route by committed instance.

4.7 Two control planes, one principle

Theorem 6 reduces every operation to coordinates of one placement problem. A production serving stack, however, does not solve that problem at one scope or one cadence: a *cluster* plane decides where work goes, and an *instance* plane decides what each engine does with the work it holds. We make this structure explicit, because the two planes operate at different cadences and admit different analyses, and because conflating them would claim a centralized global optimum no deployed system computes. Our claim is not joint optimality; it is that each plane instantiates the same residency-density principle at its own scope, with the formal guarantee living where it is both provable and operationally dominant—the instance plane.

The cluster plane: routing and flow control. A router sits in front of the fleet. For each arriving request it decides, per arrival, one of: dispatch to a chosen instance, hold in a queue, or reject (flow control). While a request waits in the router's queue the assignment is *revisable*—the router may re-assess the best instance as cluster state evolves—but once the request is *dispatched* it is pinned: it is not rerouted across instances thereafter. The cluster plane is therefore an *online assignment-with-admission* problem (requests arrive one at a time; past dispatches are irrevocable), the online-knapsack family [42], whose natural yardstick is competitive rather than approximation-vs-optimum. It is also a *multiple-knapsack* structure—each instance its own capacity—which is a different problem class from the per-instance multidimensional knapsack [40].

The instance plane: scheduling, preemption, tiering. Once a request is pinned to instance g, instance g owns it. Per epoch, g selects from its pinned candidates the resident set over its own per-axis capacities: admission/continuation, preemption, and tiered demotion, exactly the read-offs of section 4.3, now scoped to one instance. Because pinning removes cross-instance movement from this scope, the instance plane is a self-contained vector knapsack over g's axes, and theorem 8 applies per instance, with the sharper $\frac{1}{2}$ of theorem 7 in the single-binding-axis regime.

How each plane uses the common formulation. Both planes score candidates by the same urgency-weighted residency density of eq. (2); they differ only in the choice set the score ranges over. The instance plane ranges it over subsets of pinned requests under one instance's capacities (PLACEKV as defined). The cluster plane ranges the same score over candidate instances for a single arriving request—dispatch to the instance where admission yields the highest entitlement-weighted density net of cache-move cost, equivalently the instance the request least unbalances—with "queue" and "reject" as additional candidates: a request whose best feasible placement has nonpositive score everywhere (its owner's bucket is drained on the binding axis of every instance) is held or rejected rather than dispatched.

The coupling, and why we do not co-optimize. The planes couple in one direction: routing determines each instance's binding-axis profile. A router that piles axis-heterogeneous load onto one instance manufactures multi-binding epochs and erodes the single-binding-axis regime in which the sharper bound of theorem 7 applies; a router that preserves axis-homogeneity keeps each instance near one binding axis—which is what prefill/decode disaggregation does structurally. The cluster plane's objective is thus to preserve the conditions

under which the instance guarantee is tight. We deliberately do not solve the planes jointly: a global per-epoch optimum would require centralizing every instance's occupancy vector at epoch cadence, which no scalable stack does and whose guarantee would be operationally irrelevant. The honest decomposition—online assignment above, per-instance approximation below, coupled by the routing objective of keeping binding counts low—matches deployment; its empirical program (the control-plane ablation and binding-count measurement) is left to future work.

4.8 Multi-instance routing as placement

Multi-instance routing is the instance of the master reduction (theorem 6) in which the axes are indexed by instance and each request must place on a single instance's axes. We record the explicit form because heterogeneous capacities are common in practice. Now suppose there are instances $g = 1, \dots, G$, each with KV capacity K_g, prefill throughput π_g, and decode throughput δ_g. The cost of request (P, D) on instance g is

$$C^g_{\mathrm{KV}}(P, D) = \frac{P^2}{2\pi_g} + \frac{PD + D^2/2}{\delta_g}.$$

Tenant i's cluster-wide accumulated usage is

$$A_i(t) = \sum_{g=1}^{G} A_{i,g}(t), \qquad A_{i,g}(t) = \int_0^t k_{i,g}(s)\,\mathrm{d}s.$$

A global memory-share entitlement gives

$$B_i(t) = B_i(0) + \omega_i \left(\sum_g K_g \right) t - A_i(t).$$

Routing and scheduling can again be written as resident-set optimization:

$$\{S^\star_g(t)\}_{g=1}^G \in \operatorname*{arg\,max}_{S_g \in \mathcal{F}_g(t)} \left\{ \sum_{g=1}^{G} \sum_{r \in S_g} \Phi(B_{\tau(r)}(t)) \rho_{r,g}(t) - \mathrm{CacheMoveCost}(r, g) \right\}.$$

A request is routed to the instance whose selected resident set contains it. Prefix-cache locality enters through CacheMoveCost and through effective prompt miss length. If instance g already has $h_{r,g}$ prefix tokens cached, then the effective prefill miss may be $P - h_{r,g}$.

Theorem 9 (Cluster-level rate-plus-burst fairness)**.** *If tenant i's cluster-level KV-time bucket is enforced with entitlement $\omega_i \sum_g K_g$, bucket size $B_i^{\max}$, and overdraft H_i, then*

$$A_i(t_2) - A_i(t_1) \le \omega_i \left(\sum_g K_g \right) (t_2 - t_1) + B_i^{\max} + H_i.$$

Proof. Apply the single-instance bucket argument to the cluster aggregate. Setting $\widehat{K} = \sum_g K_g$ and $\widehat{A}_i(t) = \sum_g A_{i,g}(t)$, the bucket dynamics

$$B_i(t_2) = B_i(t_1) + \omega_i \widehat{K}(t_2 - t_1) - \big(\widehat{A}_i(t_2) - \widehat{A}_i(t_1)\big) - L(t_1, t_2)$$

are formally identical to the single-instance case, with $L \ge 0$ the cluster-aggregate refill loss. Saturation gives $B_i(t_1) \le B_i^{\max}$ and the overdraft rule gives $B_i(t_2) \ge -H_i$, yielding the stated bound. □

Multi-instance routing therefore strengthens the abstraction: routing is not a separate load-balancing problem; it is the first stage of the same KV-time resident-set selection problem, executed at the cluster plane of section 4.7—assessment continuous while a request waits at the router, commitment irrevocable on dispatch. In the vector framework (section 2.2) heterogeneous instances are simply axes with different capacities and exchange rates, and the entitlement caps protect each tenant's dominant share regardless of which instance it runs on.

5 Experimental Results

Everything so far establishes that residency is the right object and what can be proved on it. The remaining question is whether it is physically real: does the mirage appear under realistic demand, does residency entitlement substantially reduce it, do the prior fairness meters fail in exactly the way the projection lemma predicts, does the arrival-curve bound we proved actually hold, and is the time-integral that distinguishes our mechanism from an instantaneous cap doing observable work? We answer these in two stages: a scalar campaign of three experiments establishing that the mirage is real and that residency entitlement removes it where each prior meter cannot, and a vector campaign whose decisive finding is a dissociation no scalar meter can produce—each experiment tied to a specific theorem and phrased so that a named observation would refute it. Residency is instrumented directly from paged-KV state, not fit; the closed form is never computed as the meter—enforcement drains buckets on measured occupancy throughout.

Scope of this evaluation. We are explicit about what these experiments do and do not establish. The evaluation proceeds in two stages, both on a discrete-event simulator whose step latencies are fit from profiled microbenchmarks. The first stage (section 5.1) validates the *scalar* ($m = 1$) residency projection—a single undifferentiated KV pool—which is the regime where the central pathology is sharpest and most directly measurable, and where each theorem maps to a clean falsifiable test, so it is the right place to establish that the abstraction is physically real before generalizing. The second stage (section 5.2) tests the object the paper is ultimately about: the residency *vector*, on a two-axis prefill/decode-disaggregated configuration that exercises the per-axis metering, time-integration, and integrate-then-combine ordering a scalar cannot reach. What both stages share, and what we are deliberately *not* claiming, is a real-system study: we do not run on a production engine, and the one quantity the mechanism depends on—low-overhead per-tenant occupancy metering—we demonstrate on a real stack only as future work. The closed form is not at issue here: it is an idealized lens, never computed in the loop, so no real-hardware calibration of it gates any claim. The reader should read section 5 as confirming both the scalar special case and the vector object on a profiled simulator, with the bridge to deployment scoped but not yet built.

Experiments run on a discrete-event, iteration-level simulator of a continuous-batching engine (vLLM/Sarathi-Serve style) with paged KV cache of capacity K, chunked prefill, batch- and length-dependent step latency (not constant π, δ), preemption with resume cost, and per-tenant accounting; step-latency tables are fit from profiled microbenchmarks for a representative model (Llama 3.1-8B on an H100, TP = 1) so throughput degrades realistically with batch size and context length. Unless noted, $K = 24{,}576$ blocks of 16 tokens, per-tenant entitlement $\omega_i = 0.45$ (headroom $\eta = 0.9$), bucket depth $\beta = 10\,\text{s}$, and overdraft floor $H = 0$; each cell is five seeds with percentile-bootstrap 95% confidence intervals ($n_{\text{boot}} = 2000$), reported throughout. The schedulers run on a common executor under realistic preemption, differing only in what they meter and how they choose victims, which puts them on equal footing. Two preemption paths are active. All seven schedulers inherit the engine's vLLM-style *decode-growth* preemption: when a running request cannot allocate a block for its next decode step, the engine evicts the tail of the running batch (tenancy-blind), releasing KV and re-enqueuing the victim to re-prefill; this fires on the order of 10^3 times per 600 s run for every scheduler, so no baseline is a no-preemption strawman. KVtime additionally preempts *proactively at admission*: when a waiting candidate it wishes to admit cannot allocate, its density rule $\Phi(B_i)/k_r$ ($\Phi(B) = \max(0, B)$) selects which incumbents to evict, entitlement-aware rather than tail-of-batch. KVtime's distinctiveness is therefore not *whether* eviction happens but *who* is evicted (density-ordered, entitlement-aware) and *when* (proactively at admission); the cost of this churn—evicted requests re-prefill from scratch, inflating their TTFT—is included in the TTFT and coverage numbers we report, not netted out. KV-quota is the one scheduler that effectively never preempts: its hard admission cap ($k_i \le \omega_i K$) and enqueue-time serveability check prevent allocation from ever failing, so it prevents over-commitment rather than curing it—the admission-control-versus-preemption contrast the mechanism of section 4.3 draws. Theorems 2 and 3 are stated for strict bucket regulation with overdraft floor H_i; our deployment realizes this through the combined admission gate and density-ordered eviction, and an over-entitlement (non-conformant) tenant may still exceed its own bucket slack because tail-of-batch eviction is tenancy-blind and KVtime evicts to honor the *conformant* tenant's entitlement rather than to claw the heavy tenant exactly back to its bucket. The guarantees we test below are thus the conformant tenant's service floor (theorem 3) and its isolation (theorem 2). The central fairness metric is the residency-share ratio $\rho_{\text{mt}} = \max_i s_i / \min_i s_i$, the worst-case disparity in time-integrated

KV residency across tenants, where tenant i's residency share $s_i = A_i(T)/(K\,T_{\text{active}})$ is the area under its KV-occupancy curve (token-seconds) as a fraction of total cache capacity-time. Under equal entitlements ($\omega_i = 0.45$ for both tenants here) perfect fairness is equal shares, so $\rho_{\text{mt}} = 1$ is exact parity and larger is more skewed; it is reported alongside the conformant tenant's p95 time-to-first-token (TTFT) and its coverage (fraction of requests served). A word on what placement theory this exercises. The unified density rule of section 4.3 is not merely proved and then set aside: it is the operative mechanism here, governing admission, continuation, and proactive preemption/eviction through the single score $\Phi(B_i)/k_r$. The master reduction's central claim—that these are not separate controls but one selection problem over residency—is therefore *exercised*, not just asserted: the same rule decides who is admitted and who is evicted, in this scalar campaign and in the vector campaign below alike. What this submission does *not* exercise is the two controls that need topology these single-instance runs lack—cluster-plane *routing* and *tiering*—and the placement *approximation guarantee* itself (theorems 7 and 8): the binding-axis count and the leave-one-out value of each control are observability the runs do not yet report. These—a placement control-plane ablation, and a tiering study extending the transfer-axis result to demotion across tiers—we leave to future work. The ledger is thus exact: admission, continuation, and preemption run under the one unified rule; routing, tiering, and the approximation bound remain theory with their empirical arm named.

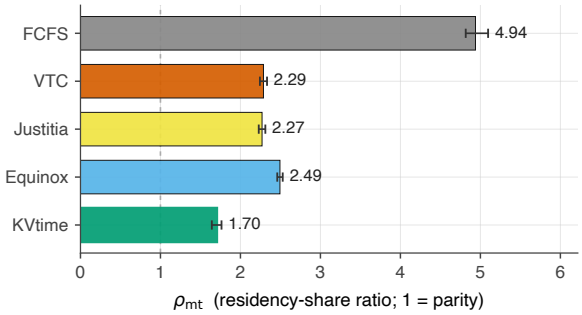
5.1 Scalar validation: the mirage, the fix, and the baseline wall

Experiment 1: the mirage is a phenomenon of demand asymmetry (tests proposition 2). Proposition 2 is an existence result: at equal mean prompt length, a token meter that equalizes token service can leave residency arbitrarily skewed. Its hypothesis—equal means—is exactly what makes it a clean separation: demand is held equal, so any residency skew is attributable to the meter alone and not to one tenant simply asking for more. The size of the separation, however, scales with residency variance *relative to capacity*: the construction drives the gap by concentrating one tenant's prompt-length distribution, and the cache absorbs variance up to its size. At the large K of current accelerators ($K = 24{,}576$ blocks in our configuration), the cache therefore acts as a coarse equalizer under *symmetric* load, and the equal-mean separation, while present and theory-predicted, is quantitatively modest: across arrival rates we measure $\rho_{\text{mt}} \approx 1.0$ for all schedulers, residency-aware and residency-blind alike. This is not evidence against proposition 2—the theorem is an existence statement at unbounded variance ratio, proved independently of K—but a measurement of where on the variance/capacity curve a realistic symmetric workload sits.

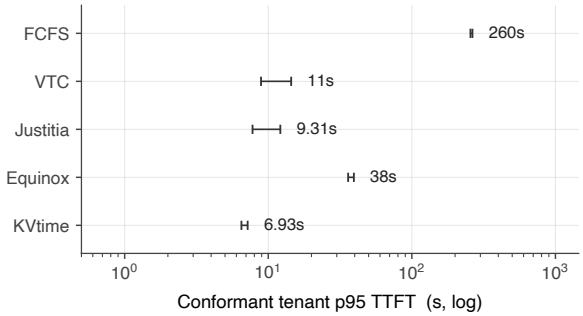
The operative regime is *demand asymmetry*, ubiquitous in production multi-tenancy (free versus paid tiers, chat versus retrieval-augmented workloads, bursty versus steady clients), which amplifies the meter's blindness into a large, robust effect without any parameter search. The pivot is theory-guided: the same variance/capacity dependence that makes the symmetric case modest predicts that asymmetry—which raises one tenant's residency variance and demand together—makes it severe. Experiment 2 confirms this directly.

Experiment 2: residency entitlement substantially reduces the skew and protects the conformant tenant (tests theorems 2 and 3). Under demand asymmetry the mirage becomes a conformant-tenant catastrophe, and this is where residency entitlement earns its place. We evaluate two production-realistic asymmetries. **W5** is a free-tier-versus-heavy-customer scenario: a 1:5 arrival-rate imbalance with equal-mean prompts, so the only difference between tenants is how often they ask. **W7** is a chat-versus-RAG scenario: equal arrival rates but a 16× mean-prompt-length imbalance, so the only difference is how much memory each request holds. Both isolate a single asymmetric knob; in both, the residency-blind schedulers starve the conformant tenant, and residency entitlement (KVtime) protects it. We compare against FCFS (no meter) and VTC [17], the token-time meter that is the strongest residency-blind incumbent.

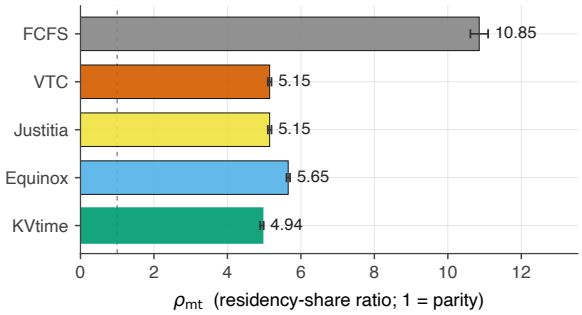
Figure 1 reports the result on both fairness axes. Under W5, FCFS lets the heavy tenant drive the residency-share ratio to $\rho_{\text{mt}} = 4.94$ $[4.82, 5.10]$ and inflates the conformant tenant's p95 TTFT to 260 s while serving only 51% of its requests. VTC, metering token-time, tightens residency to $\rho_{\text{mt}} = 2.29$ $[2.24, 2.33]$ and TTFT to 11.0 s—a real improvement, but it leaves the conformant tenant more than twice over its entitled residency. KVtime, metering residency directly, holds $\rho_{\text{mt}} = 1.70$ $[1.65, 1.76]$ and p95 TTFT of 6.93 s at 96% coverage: a 2.9× improvement in residency fairness and a 38× improvement in conformant tail latency over FCFS. Under W7 the prompt-length asymmetry makes long requests dominate residency regardless of meter, so absolute ρ_{mt} values are larger, but the ordering and the latency story are stark:



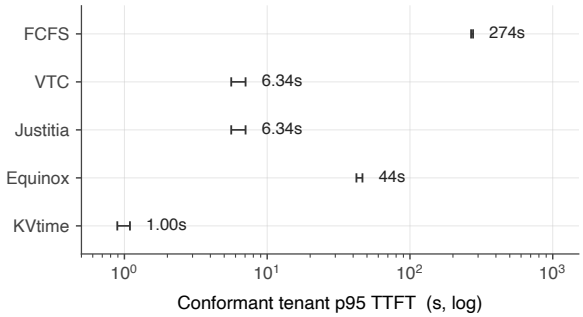
(a) W5: residency fairness



(b) W5: conformant p95 TTFT



(c) W7: residency fairness



(d) W7: conformant p95 TTFT

Figure 1: **Residency entitlement substantially reduces residency skew and protects conformant tail latency, under both kinds of demand asymmetry** (five seeds, bootstrap 95% CI; right column is conformant p95 TTFT on a log axis; dashed line in the left column is parity $\rho_{\text{mt}} = 1$). Top: W5, a 5× arrival-rate asymmetry. Bottom: W7, a 16× prompt-length asymmetry. The residency-blind baselines (FCFS, the token-time meter VTC [17], and the two closest prior fairness meters Justitia [18] and Equinox [29]) leave the conformant tenant skewed in residency and stranded in the tail—hundreds of seconds under FCFS; Justitia tracks VTC and Equinox lands slightly worse than VTC, neither closing the gap to KVtime. KVtime, metering residency directly, drives ρ_{mt} toward parity and the conformant tenant's tail latency into the single-digit seconds (38× faster than FCFS on W5, 274× on W7), at 95–96% coverage and held utilization.

FCFS reaches $\rho_{\text{mt}} = 10.85$ $[10.62, 11.09]$ with conformant p95 TTFT of 274 s at 48% coverage; KVtime holds $\rho_{\text{mt}} = 4.94$ $[4.89, 4.99]$ with a p95 TTFT of 1.00 *s* at 95% coverage—a 274× tail-latency improvement while serving twice the conformant requests. The fairness gain is not bought by idling the cache: aggregate utilization is held throughout (cache-idle fraction within a few points of FCFS in every cell).

This is the result the theory predicts: the conformant tenant is protected because the meter sees the resource the system actually runs out of. The mechanism behind the protection—that the time-integral is doing the work, and that the bound of theorem 3 holds while it does—is isolated below.

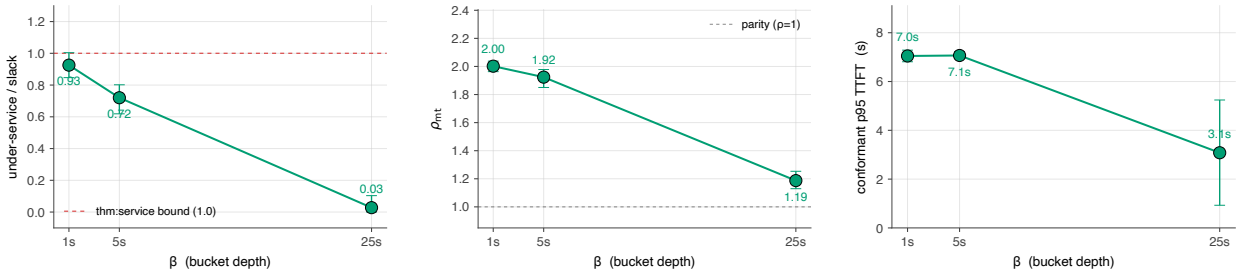
The two closest prior fairness meters land between FCFS and KVtime—and neither closes the gap. Beyond the token meter VTC, we run the two nearest prior fairness designs as meters under the same admission mechanism (executor, workload, seeds, and hardware fixed), so the projection lemma's placement of them becomes measured rather than asserted. *Justitia* [18] meters scalar KV-token-time ($\pi \to \infty$, $\delta = 1$ in our charge); *Equinox* [29] blends a weighted-token/latency user counter with an aggregate-utilization resource counter, fed a perfect oracle for the metrics its predictor would estimate. The measured outcome refines the naive prediction in two honest ways. First, Justitia is *byte-identical to VTC on W7* ($\rho_{\text{mt}} = 5.15$ on both, every seed): under extreme prompt-length asymmetry the long-prompt tenant dominates both token-time and KV-token-time, so the argmin ordering is identical and the prefill-build-up term does not change *who* is served; the two scalar meters diverge instead under *rate* asymmetry (W5, where Justitia is marginally fairer

than VTC at 2.27 vs 2.29). The honest claim is therefore the structural one—KVtime strictly generalizes Justitia, and both token-family meters stay residency-skewed—rather than a long-context mechanism. Second, Equinox lands *above* VTC on residency skew, not between VTC and KVtime ($\rho_{\text{mt}} = 5.65$ vs 5.15 on W7; 2.49 vs 2.29 on W5): even granted an oracle predictor, blending a utilization coordinate into the user counter orders service slightly worse for residency parity than the token-time counter alone—consistent with proposition 2, since neither coordinate is the held resource. The load-bearing claim survives in full with five-seed CIs: every residency-blind meter, including the two closest prior fairness meters, leaves residency materially more skewed than KVtime, which alone drives the conformant tenant to single-digit-second tails.

The burst depth β is mechanistically necessary, and the service bound holds. KVtime's distinguishing feature over an instantaneous cap is that it integrates residency over time and lets a tenant bank credit up to a burst depth β. To show this integral is not stylistic, we sweep $\beta \in \{1, 5, 25\}$ s on W5 (fig. 2). Two things move together. First, fairness sharpens monotonically: ρ_{mt} falls from 2.00 $[1.94, 2.07]$ at $\beta = 1$ s to 1.19 $[1.16, 1.22]$ at $\beta = 25$ s, and the conformant tenant's p95 TTFT falls from 7.0 s to 3.1 s—deeper buckets let banked credit absorb the heavy tenant's surges precisely where prompt-length variance produces requests near the entitlement, which a shallow bucket with little banked credit cannot. Second, this responsiveness is not bought by loosening the guarantee. Theorem 3 bounds a backlogged conformant tenant's under-service by the aggregate competing slack; we measure the realized ratio of under-service to that slack and find it holds in all five seeds at every β, tightening from 0.93 $[0.89, 0.96]$ at $\beta = 1$ s to 0.03 $[0.01, 0.05]$ at $\beta = 25$ s. The bound is informative rather than vacuous, and the conformant tenant's under-service becomes essentially zero exactly where the bucket is deepest. The two-sided guarantee's isolation side (theorem 2) holds for the conformant tenant throughout; the heavy non-conformant tenant exceeds its own bucket slack, as expected—KVtime's density-ordered eviction acts to protect the conformant tenant's entitlement rather than to claw the heavy tenant exactly back to its bucket, and the theorem speaks only to bucket-feasible tenants.

Operator guidance for (β, H). The sweep doubles as a tuning rule. Bucket depth β trades *burst tolerance* against *convergence to entitlement*: deeper buckets bank more credit, absorbing legitimate surges (fairness sharpened from 2.00 to 1.19, conformant tail from 7.0 to 3.1 s across the sweep) at the cost of letting a tenant run further from its long-run share before regulation bites; the isolation bound of theorem 2 degrades exactly linearly in β, so the operator's exposure is explicit, not hidden. A practical default is to size β to the largest *legitimate* single-request residency a tenant should serve without rejection (the coverage analysis of Experiment 3b gives the tail probability this protects), and to keep the overdraft H at zero unless a tenant class explicitly purchases oversized-request headroom—H widens both sides of the two-sided guarantee symmetrically. Within the sweep's range the service bound held at every setting, so the tuning choice moves the fairness/burst trade-off, not the guarantee's validity. Urgency u completes the (ω, β, u) triple and is the SLO knob proper: because it reorders only within the bucket-feasible set and cannot move long-run share (theorem 4 and proposition 10), it can be set from deadline tightness alone—ω from the contracted share, β from the tolerated burst and oversized-request tail, u from the SLO class—with no risk that an aggressive SLO class becomes a side channel on share.

Experiment 3a: the baseline wall—every prior meter is the wrong axis (tests lemma 1). Lemma 1 states that each standard fairness meter is residency with one coordinate collapsed, and therefore cannot certify residency parity. We make this concrete by running eight governance layers on the *same* executor, workload (W7), seeds, and hardware, differing only in the meter: request-count round robin (unit-count collapse), decode-token fairness (output-axis collapse), head-of-line wait accounting (endogenous-time collapse), FCFS (no meter), VTC [17] (token-time collapse), Justitia [18] (KV-token-time, the build-up-free single-axis case), Equinox [29] (token-time $\times$ utilization projection), and KVtime (residency). Figure 3 reports three things side by side. Each projection meter *succeeds* on its own coordinate (fig. 3a): request-RR equalizes per-tenant request count to a completions ratio of 1.01, decode-token to 0.98, HOL-wait to 1.00. Yet on residency (fig. 3b) the residency-blind schedulers cluster well above parity—the count/wait family at $\rho_{\text{mt}} \approx 11$–$14$, and the token-time family (VTC, Justitia, Equinox) at ≈ 5–6—and on the conformant tenant's tail latency (fig. 3c) the count/wait family sits indistinguishably with FCFS at 268–275 s. Meeting your own fairness goal does not translate to residency fairness: this is the projection lemma in one figure.



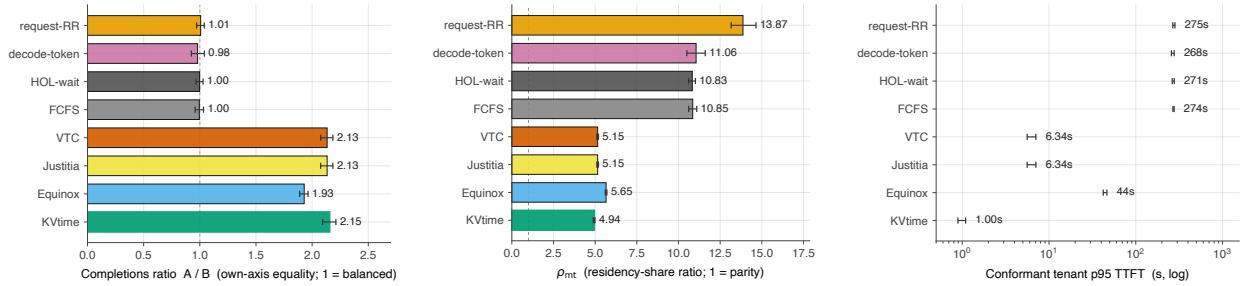
(a) service bound (theorem 3) (b) residency fairness (c) conformant p95 TTFT

Figure 2: **The time-integral does observable work, and the service bound holds while it does** (KVtime on W5; five seeds, bootstrap 95% CI). Sweeping bucket depth β from 1 to 25 s: (a) the ratio of conformant under-service to aggregate competing slack stays below the theorem 3 bound of 1.0 in all five seeds at every β, falling to 0.03 at $\beta = 25$ s; (b) ρ_{mt} sharpens from 2.00 toward parity at 1.19; (c) conformant p95 TTFT falls from 7.0 to 3.1 s. Deeper buckets bank more credit to absorb the heavy tenant's variance—ablating β toward zero degenerates the mechanism to a conservative, near-instantaneous cap with no banked credit.

The sharpest case is request-count round robin. Equalizing per-tenant request count—the most intuitive notion of "fair shares"—produces $\rho_{\text{mt}} = 13.87$ [13.16, 14.65], *more* skewed than doing nothing at all (FCFS's 10.85). Equalizing the wrong coordinate is not merely less helpful than residency metering; under prompt-length asymmetry it is actively worse than no meter, because it hands the heavy tenant equal request count while each of its requests holds far more memory for far longer. The token-time family clusters next: VTC and Justitia are *byte-identical* on this workload ($\rho_{\text{mt}} = 5.15$), since under extreme prompt asymmetry token-time and KV-token-time induce the same service order, and Equinox sits just above them at 5.65. KVtime ($\rho_{\text{mt}} = 4.94$) is the only scheduler that improves on the entire token-time family, and it does so by being intentionally *unequal* on completions (ratio ≈ 2.1)—which is exactly what residency parity requires when one tenant's requests are sixteen times larger. No projection meter both serves all requests and matches KVtime on residency; the lemma's refutation condition is not met.

Experiment 3b: the coverage failure of strict caps (tests lemma 1, sup-collapse). The wall isolates one further consequence of the sup-collapse projection: the instantaneous quota $k_i(t) \leq \omega_i K$ enforced at admission. Beyond missing duration, a strict instantaneous cap has no escape hatch for a single request whose own footprint exceeds the per-tenant entitlement $\omega_i K$—such a request can never be admitted, at any cache state, because admitting it alone would violate the cap. We measure this on a heavy-tailed prompt distribution under tight contention ($K = 2{,}048$, so $\omega K = 921$ blocks, a small multiple of the median prompt), sweeping the lognormal scale $\sigma \in \{2.0, 2.5, 3.0\}$ of the heavy tenant. We compare KV-quota (the strict cap, enforced as reject-on-overflow, the faithful reading of "$k_i \leq \omega_i K$ at all times") against KVtime, whose admission gate tests bucket balance rather than request size, so for any $\beta > 0$ an oversized request is admitted against positive balance and repaid over time.

The strict cap drops exactly the tail the analysis predicts (fig. 4a). The closed-form probability that a single prefill exceeds ωK under the lognormal tenant is 4.27%, 5.61%, 6.63% at the three σ; KV-quota's measured drop rate is 4.20% [3.84, 4.61], 5.50% [5.19, 5.81], 6.66% [6.34, 6.99]—tracking the analytical curve within bootstrap noise at every point. KVtime drops zero across all σ. The outcome breakdown at $\sigma = 2.5$ (fig. 4b) shows the trade the cap is making: KV-quota completes 64% of the heavy tenant's requests but rejects 5.5% outright, buying predictability up front by refusing the large requests; KVtime drops none. This is the cleanest statement of why residency must be *integrated over time* rather than *capped instantaneously*—the integral, banked as burst, is exactly what lets a tenant serve a request larger than its instantaneous share while remaining fair in the long run. The cap's refutation condition (strict cap serves the oversized requests, or KVtime drops them) is not met.



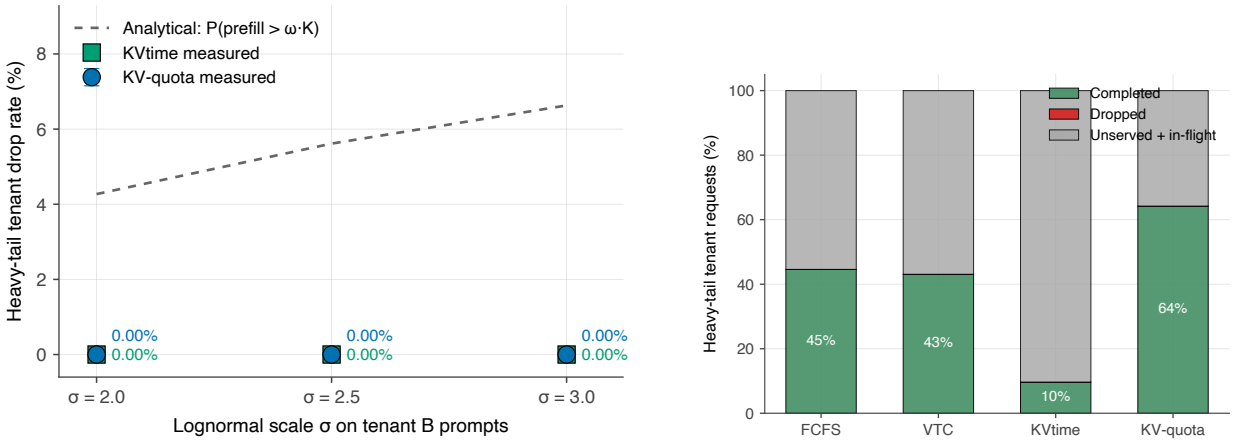
(a) own-coordinate equality (b) residency fairness (c) conformant p95 TTFT

Figure 3: **The projection lemma made concrete: meeting your own fairness goal does not produce residency fairness** (eight schedulers on W7; five seeds, bootstrap 95% CI; (c) on a log axis). (a) Every projection meter equalizes its own coordinate—request-RR matches request counts (1.01), decode-token matches output tokens (0.98), HOL-wait matches head-of-line wait (1.00). (b) Yet the count/wait family leaves $\rho_{\text{mt}} \approx 11$–$14$ (request-RR's unit-count collapse is *worse than FCFS*, 13.87 vs 10.85, the wrong-axis penalty), and the token-time family (VTC, Justitia, Equinox) sits at ≈ 5–6—Justitia byte-identical to VTC, Equinox just above. (c) The count/wait family strands the conformant tenant at 268–275 s. Only KVtime breaks fully from every cluster, at the cost of intentionally unequal completions—the price of residency parity under prompt-length asymmetry.

5.2 From the scalar special case to the vector: the object the paper is about

Everything above validates the scalar ($m = 1$) projection—the single undifferentiated KV pool—where the central pathology is sharpest and each theorem maps to one falsifiable test. But the scalar is the special case; residency is a vector *(definition 4), and the claims that distinguish this paper from every prior meter live on the axes a scalar cannot see: that collapsing residency to one bucket forfeits dominant-share fairness (proposition 3, lemma 1), and that the order in which one combines the axes and integrates over time is itself load-bearing (theorem 2). No scalar result can establish these. We now test them directly on a two-axis configuration, and the sharpest finding is not that vector KVtime wins everywhere—it does not, and the place it "loses" is exactly where the theory says it must—but that the metric on which it wins is the one the theory identifies as correct.*

Setup. We extend the simulator to meter two residency axes under **prefill/decode disaggregation**: axis 1 is prefill-GPU KV residency (HBM held during the compute-bound build-up), axis 2 is decode-GPU KV residency (HBM held during the memory-bound tail), each with its own effective capacity ($K_P = 2{,}048$, $K_D = 22{,}528$ blocks; per-instance 24,576 blocks at 16 tokens) and its own per-tenant bucket; every request runs prefill on the P-GPU then hands its KV to the D-GPU, the hand-off treated as an instantaneous, non-overlapping transfer of occupancy from axis 1 to axis 2 (no double-counting). Two tenants concentrate on *different* axes: tenant A prefill-heavy (long prompts, short generations), tenant B decode-heavy (short prompts, long generations), each tenant's dominant axis being the one the other barely touches; entitlements are equal on each axis ($\omega = 0.45$). We compare five governance layers, each isolating one missing ingredient: **Vector KVtime** (per-axis, time-integrated, integrate-then-combine, rate-plus-burst—definition 4); **Scalar KVtime** (the axis-collapsed $m = 1$ projection, time-integrated but not per-axis); **static DRF** [24] (per-axis but on instantaneous occupancy, not time-integrated); **SDRF-on-occupancy** [26] (time-aware and axis-aware but composing in the opposite order, combine-then-integrate, regulating by share-averaging rather than rate-plus-burst—the strongest time-aware foil); and **FCFS** (no meter, floor). The campaign is five schedulers × three workloads × five seeds = 75 cells, 600 s each (30 s warmup), percentile-bootstrap 95% CIs ($n_{\text{boot}} = 2000$); $\beta = 10$ s, $H = 0$, sliding window 10 s. The reported metric is each tenant's *dominant* residency share (max-over-axes time-integrated residency) and the dominant-share disparity ρ_{dom} across tenants; $\rho_{\text{dom}} = 1$ is parity. As in the scalar campaign, Vector KVtime is run as the unified density rule of section 4.3—now scored per axis—governing admission and proactive eviction across both buckets from one selection, so the master reduction's collapse of these controls is exercised on the vector object, not only the



(a) measured vs. analytical drop rate

(b) outcome breakdown ($\sigma = 2.5$)

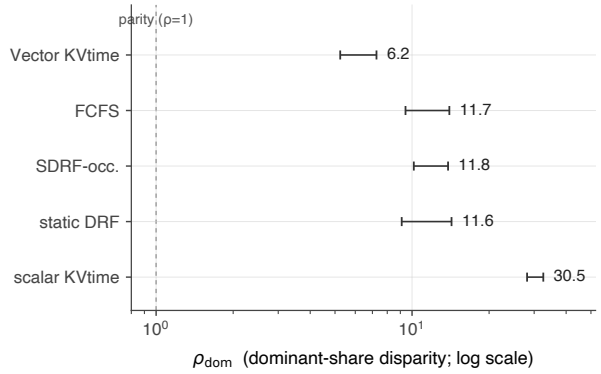
Figure 4: **A strict instantaneous cap structurally drops the heavy-tail requests it cannot fit; the time-integral does not** ($K = 2{,}048$, heavy-tail tenant, five seeds, bootstrap 95% CI). (a) KV-quota's measured drop rate tracks the analytical tail probability $P(\text{prefill} > \omega K)$ within CI across the lognormal scale σ (4.20/5.50/6.66% measured vs. 4.27/5.61/6.63% predicted); KVtime drops zero. (b) At $\sigma = 2.5$, KV-quota completes more of the heavy tenant's work but rejects 5.5% as structurally unadmittable, trading reach for up-front predictability; KVtime banks the oversized requests against burst credit and drops none.

scalar one. Residency conservation holds exactly in every cell (0 violations /75).

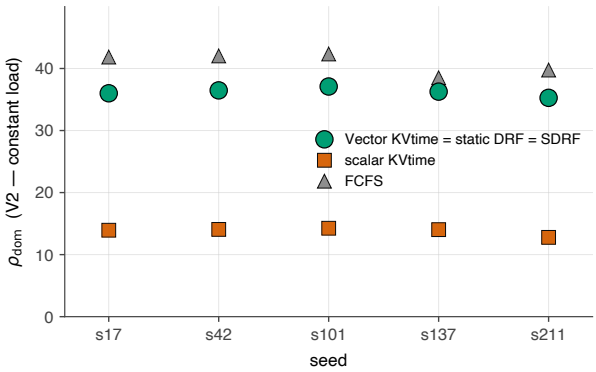
V1, bursty—the per-axis win, and the cost of collapsing it. On an axis-separated bursty workload (concentrated per-axis demand, CV > 1), vector KVtime attains the lowest disparity, $\rho_{\text{dom}} = 6.24$ $[5.24, 7.26]$, and its CI lies *entirely below every baseline's*—its upper bound (7.26) sits beneath the next-best lower bound (static DRF, 9.11), so the separation is statistical, not a point estimate (fig. 5, left). The cautionary control is scalar KVtime, *worst of all five* at $\rho_{\text{dom}} = 30.47$ $[28.15, 32.57]$, roughly $5\times$ vector's: collapsing the two axes into one bucket lets the decode-heavy tenant's large decode-axis residency be "paid for" by its near-zero prefill-axis usage, subsidizing itself at the prefill-heavy tenant's expense. This is proposition 3 and lemma 1 made physical—the axis collapse is not a modeling nicety but a $5\times$ fairness regression.

V2, near-constant—the exact tie the theory demands. Under axis-separated but near-constant occupancy (CV ≈ 1), the three regulated schedulers—vector KVtime, static DRF, SDRF—produce *byte-identical* ρ_{dom} at every seed: not within CI but equal to the last printed digit (e.g. 37.0961 at seed 101, 35.2699 at seed 211), one marker series standing for all three (fig. 5, right). This is the strongest possible confirmation of proposition 8: when occupancy is time-invariant, the integral and burst terms that distinguish vector KVtime contribute nothing, and integrate-then-combine, combine-then-integrate, and the instantaneous snapshot all collapse to the same admission/eviction decisions. The consequence for the narrative is precise: vector KVtime's V1 advantage is specifically a *burstiness* advantage, and this figure shows it vanishing honestly where the theory says it must—rather than papering over the regime where the mechanisms coincide.

V3, axis-migration—the dissociation, and the paper's sharpest empirical claim. The decisive experiment drives a workload in which the tenants' roles *flip axis* at $t = 300\,\text{s}$: the prefill-heavy and decode-heavy tenants swap, so each tenant's instantaneous dominant axis migrates over the horizon—precisely the regime where the non-commutation $\int \max \geq \max \int$ separates combine-then-integrate from integrate-then-combine (section 6). Here vector KVtime does *not* win the headline number. On long-run ρ_{dom}, SDRF wins: 1.80 $[1.70, 1.90]$ against vector's 2.97 $[2.73, 3.21]$. But on *per-window* isolation—the 99th-percentile dominant-share excursion of the axis-migrating tenant over sliding 10 s windows—the ranking *inverts*: vector KVtime holds 0.31 $[0.25, 0.37]$ while SDRF reaches 0.54 $[0.53, 0.55]$, with CIs non-overlapping in both directions



(a) V1 bursty: vector KVtime bounds ρ_{dom} below all baselines



(b) V2 constant: the regulated trio coincides byte-identically

Figure 5: **The per-axis win (V1) and the exact recovery (V2).** (a) On bursty axis-separated load, vector KVtime's 95% CI lies entirely below every baseline (log axis; dashed line is parity); scalar KVtime is worst by $\sim 5\times$, the cross-axis subsidy of proposition 3. (b) On near-constant load, vector KVtime, static DRF, and SDRF produce identical ρ_{dom} at every seed (one green series stands for all three), confirming proposition 8: vector's advantage is specific to burstiness. Five seeds, bootstrap 95% CI.

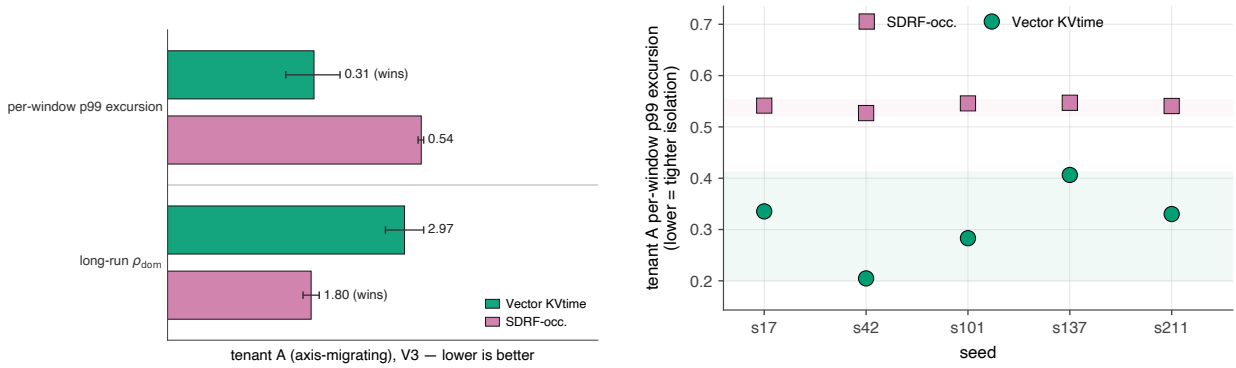
Table 1: **V3 axis-migration: the dissociation.** For the axis-migrating tenant A, the long-run and per-window metrics rank the two schedulers oppositely, CIs non-overlapping in both directions (five seeds, bootstrap 95% CI; lower is better).

Metric (tenant A)	Vector KVtime	SDRF-occ.	winner
per-window p99 excursion	0.31 $[0.25, 0.37]$	0.54 $[0.53, 0.55]$	Vector
long-run ρ_{dom}	2.97 $[2.73, 3.21]$	1.80 $[1.70, 1.90]$	SDRF

(table 1, fig. 6 left). The per-seed strip (fig. 6 right) shows the per-window win is consistent across all five seeds: every vector point sits below every SDRF point, so the effect is structural, not an averaging artifact.

This inversion is the experiment's contribution, and the reason it is more compelling than a clean sweep would be. SDRF, by collapsing the axes to a scalar dominant share *before* integrating, discards the per-axis history and so charges the migrating tenant for whichever axis is its momentary peak—which, smoothed over the run, yields a *lower long-run average* but *larger within-window excursions*, because it has no per-axis bucket to bound the transient. Vector KVtime, integrating each axis separately before combining, prices the sustained load on each resource and bounds the excursion through the per-axis bucket of theorem 2—at the cost of a higher long-run scalar that reflects real, bounded transient load rather than averaging it away. The lesson is not "our method wins" but the stronger and more useful claim that **long-run dominant share is an insufficient fairness metric for axis-migration workloads**: a scheduler can win it while permitting exactly the transient unfairness a per-axis entitlement exists to prevent. The honest headline against SDRF is therefore the one the plan predicted—comparable (here, worse) long-run dominant share, but tighter per-window isolation, plus the service floor and tunable burst SDRF structurally cannot offer—and it is sharper *because* vector KVtime concedes the averaged number.

What the vector campaign establishes. The three workloads decompose *why* the mechanism works rather than merely that it wins: per-axis metering (V1, against scalar collapse), the exact boundary of its advantage (V2, the constant-load tie of proposition 8), and the integrate-then-combine ordering (V3, the dissociation). Each baseline isolates one ingredient, and the refutation conditions stated for this experiment are not met: scalar KVtime does not equalize dominant shares; vector KVtime does, under burst; static DRF ties only on constant load; and SDRF matches vector on long-run share *yet loses per-window isolation*, so the rate-plus-burst structure demonstrably buys something share-averaging does not.



(a) the inversion: vector wins per-window, SDRF wins long-run

(b) per-seed: every vector point below every SDRF point

Figure 6: **The dissociation on axis-migration load (V3).** (a) The two fairness metrics for the axis-migrating tenant A rank vector KVtime and SDRF oppositely: vector wins per-window p99 excursion (tighter transient isolation), SDRF wins long-run ρ_{dom}; bars are normalized within each metric band with raw values labeled, so the inversion is explicit. (b) Per-seed per-window p99 excursion: every vector point sits below every SDRF point, so the isolation win is consistent seed-by-seed, not an averaging artifact. The takeaway: long-run dominant share alone is insufficient for axis-migration load—the discriminating metric is per-window excursion, exactly where the integrate-then-combine ordering of theorem 2 does its work.

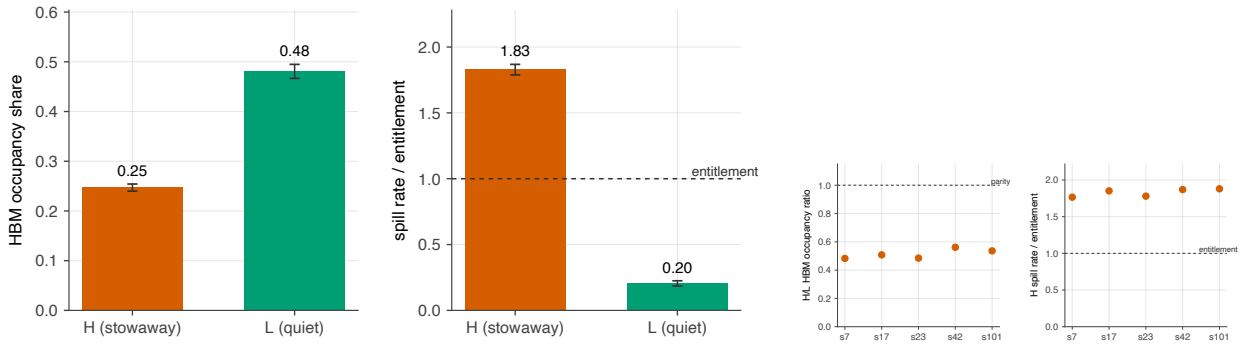
5.3 The transfer axis: stock-light, flow-heavy, and the irreducible blindness of capacity meters

The vector campaign exercised capacity axes. The stock/flow distinction (section 2.2) claims something a capacity-only generalization cannot capture: transfer is a flow*, residency a* stock*, and the two are not functions of one another, so a tenant can hold little memory while monopolizing the link that moves it. We make that sentence a measured fact, and pair it with the structural reason no capacity meter can ever see it (proposition 9).*

Setup. A single serving instance (Llama-3.1-8B, H100, TP= 1, 4000 HBM blocks) with two KV tiers: GPU HBM (a capacity axis) and a node-local NVMe spill tier reached over a shared bus at the *profiled sustained* GPU↔NVMe rate $C_{\text{BW}} = 2.0\,\text{GB/s}$—not the datasheet peak, which spill never reaches. Two tenants with equal entitlement ($\omega = 0.45$): a *stowaway* H (long 8192-token prompt spilled to NVMe, very short 8-token output—it completes fast, holding *little* time-averaged HBM but driving a *high* spill rate) and a *quiet* L (short 256-token prompt, long 1024-token decode—it holds modest KV resident for a long time and spills little). We report each tenant's time-integrated HBM occupancy share (what every capacity meter sees) against its spill-rate over entitlement $\int \tau_i \, \mathrm{d}t/(\omega_i C_{\text{BW}} \Delta)$ (what the flow meter sees, the quantity theorem 2 bounds). Five seeds, bootstrap 95% CIs; the bus binds naturally (∼0.9 utilization) at the fixed profiled rate, with no budget shrunk to force it.

The dissociation. The same tenant is the *lightest* on every capacity meter and the *heaviest* on the flow meter (fig. 7, table 2). H holds 0.51× the HBM occupancy of L (0.247 vs. 0.481, CI $[0.49, 0.54]$ on the ratio)—by occupancy, H is the good citizen—yet drives 1.83× its fair share of the spill link ($[1.79, 1.87]$) while L sits far below entitlement at 0.20×. The inversion holds in every seed (fig. 7, right): H's occupancy ratio sits near 0.5 and its flow near 1.8 across all five, so the effect is structural, not an averaging artifact. The color reversal between the two panels of fig. 7—H the short bar on the left, the tall bar on the right—*is* the dissociation.

Why no capacity meter recovers. The headline is two numbers; what makes it decisive is proposition 9. A reviewer's natural objection is that a *smarter*, time-aware capacity meter would catch H—and the answer



(a) the inversion: H lightest by occupancy, heaviest by flow

(b) per-seed: stable across all five

Figure 7: **The stock/flow dissociation on a contended spill bus.** (a) The stowaway H (vermillion) is the *shortest* bar on the HBM-occupancy axis (left) yet the *tallest* on the spill-rate axis (right, above the dashed entitlement line); the quiet L (green) is the reverse. Occupancy rates H light; flow rates H the monopolist. (b) The H/L occupancy ratio ($\sim$0.5, below parity) and H's spill rate ($\sim$1.8, above entitlement) are seed-stable. Five seeds, bootstrap 95% CI.

Table 2: **Transfer-axis dissociation.** The same tenant H is light on the capacity meter and over-entitlement on the flow meter (five seeds, bootstrap 95% CI).

	Tenant H (stowaway)	Tenant L (quiet)
HBM occupancy share (capacity meter)	0.247 $[0.240, 0.254]$	0.481 $[0.467, 0.495]$
spill rate / entitlement (flow meter)	1.829 $[1.788, 1.868]$	0.204 $[0.186, 0.225]$

is that the strongest such meter, SDRF on occupancy, *provably reduces to the same occupancy-share number* on a single axis: the dominant share over one resource collapses to its lone term. The blindness is therefore structural, not an artifact of a naive meter—every capacity meter on that tier, however cleverly it integrates over time, is the identical scalar, and that scalar has no flow term. This is the mirror image of proposition 8: there the capacity meters *coincide* on the capacity axis; here they are *all equally blind* to the flow axis, because the projection onto occupancy carries no flow coordinate. Only a meter on the bandwidth axis sees H at all. (The snapshot meter, static DRF, is not merely blind but unstable on this tier—its instantaneous occupancy reading for H flips toward zero on some seeds, mild evidence that the snapshot is even less informative than the time-integral.)

What we claim, and what we do not. The result is the dissociation—H light on every capacity meter, over-entitlement on the flow meter, seed-stable—and the provable blindness behind it. This is also the precise answer to a natural deployment worry: an aggressive offload or live-migration system (e.g. Aqua-style NVLink offload, or Llumnix-style migration) could *evade* an occupancy meter by keeping a tenant's resident footprint low while driving high transfer flow—exactly the stowaway H constructs by hand. The meter that governs such a tenant is therefore not optional sophistication but a structural requirement: only the flow axis sees it, so any platform that offers cheap KV movement must meter the bandwidth that movement consumes, or it has handed tenants a blind spot. We are deliberate about scope: theorem 2 bounds the resource *share/rate*, not victim latency. On a single shared link under fixed open-loop demand, rate-bounding the stowaway relocates contention in time rather than removing its volume, so the quiet tenant's end-to-end latency need not improve, and we do not claim it does. The measured, defensible contribution is that the flow axis is the only coordinate that sees a monopolization every occupancy meter—naive or time-aware—is structurally incapable of detecting, which is precisely the stock/flow claim the vector requires.

Summary. Across the scalar campaign the headline claim holds in the form we set out to test: on workloads where token meters look fair, residency entitlements deliver materially better residency fairness

Table 3: **Efficiency under residency regulation** (five seeds, bootstrap 95% CI). Goodput is completed requests per second; completion rate is the served fraction; occupancy pressure is $\sum_i \int k_i \, \mathrm{d}t/(KT)$, an occupancy proxy (it can exceed 1 at saturation because occupancy spanning the warmup boundary is credited against post-warmup capacity-time), *not* a literal hardware utilization. On W7 (prompt-asymmetric) KVtime has the *highest* goodput of all eight schedulers; on W5 (rate-asymmetric) its lower aggregate goodput is the bounded-share guarantee throttling the rate-abusive tenant—accompanied by the conformant tenant's coverage rising from ∼0.52 to ∼0.96 (fig. 8).

WL	Scheduler	goodput (req/s)	compl. rate	occ. pressure
W5	FCFS	16.09 [15.80, 16.38]	0.509	1.021
	VTC	16.00 [15.73, 16.23]	0.506	1.026
	Justitia	15.98 [15.72, 16.23]	0.506	1.028
	Equinox	15.96 [15.70, 16.21]	0.505	1.026
	KVtime	**14.04** [13.66, 14.35]	**0.444**	**1.002**
W7	FCFS	15.25 [14.98, 15.52]	0.484	1.020
	request-RR	15.29 [15.05, 15.55]	0.485	0.951
	VTC = Justitia	21.90 [21.78, 22.02]	0.695	1.018
	Equinox	20.83 [20.72, 20.91]	0.661	1.016
	KVtime	**22.01** [21.82, 22.19]	**0.698**	**0.961**

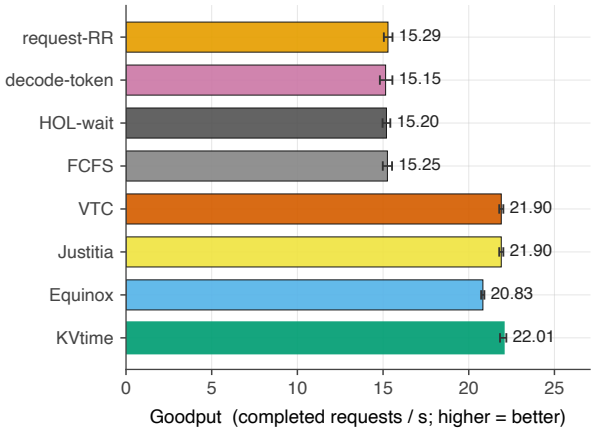
and conformant-tenant tail latency (2.9–2.2× on ρ_{mt}, 38–274× on p95 TTFT), satisfy the arrival-curve service bound we proved (theorem 3, five of five seeds at every β), and do so at held utilization—while every residency-blind meter equalizes its own coordinate yet leaves residency skewed, the strongest of them (request-count round robin) doing worse than no meter at all. The comparison is not rigged by denying the baselines preemption: every baseline runs under the same vLLM-style decode-growth eviction (on the order of 10^3 evictions per run), so their poor conformant-tenant latency reflects *tenancy-blind* victim selection, not the absence of eviction. What KVtime changes is which requests are evicted and when—density-ordered and entitlement-aware, proactively at admission—and that difference, churn cost included in the reported numbers, is what protects the conformant tenant.

5.4 The cost of fairness: goodput and utilization under residency regulation

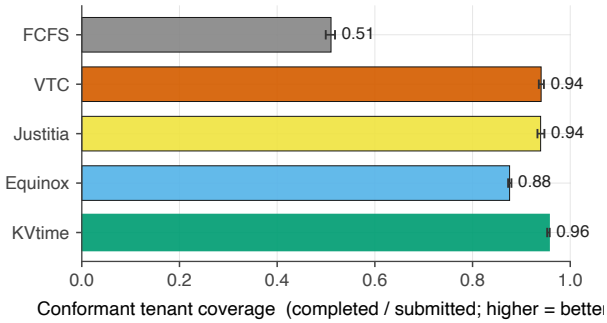
A fairness mechanism that protects the conformant tenant by throttling work would trade latency for throughput, so we report the efficiency side directly rather than asserting utilization is "held." Table 3 gives four efficiency metrics for every scheduler on both workloads. The answer is regime-dependent, and the honest version is stronger than a flat "no penalty."

No throughput tax where contention is structural (W7). Under prompt-length asymmetry, KVtime attains the highest goodput of all eight schedulers (22.0 req/s, at or above the token meters' 21.9 and far above the residency-blind cluster's ∼15.2) and the highest completion rate (0.70 vs. FCFS's 0.48). Entitlement-aware eviction completes *more* work than tenancy-blind eviction because it wastes less capacity on requests it will later re-prefill; its occupancy pressure (0.96) sits just below the token meters' saturation (∼1.02), so it delivers the most work while running the cache slightly slacker. The "held utilization" claim is, in this regime, an understatement.

Where goodput drops, the drop is the guarantee (W5). Under a 5× rate asymmetry, KVtime's aggregate goodput is lower (14.0 vs. FCFS's 16.1). This is the mechanism working, not failing: FCFS rubber-stamps the heavy tenant's flood (both tenants near 0.52 coverage), maximizing raw completions while starving the conformant tenant; KVtime instead caps the heavy tenant at its entitlement, lifting the conformant tenant's coverage to 0.96 while the abusive tenant's falls to ∼0.34. Aggregate goodput counts the throttled excess equally with protected work, so it necessarily drops—which is precisely why *per-tenant* coverage, not tenant-blind aggregate goodput, is the right efficiency lens under fairness regulation. We report both plainly rather than cherry-picking the favorable workload.



(a) W7: KVtime is the goodput leader



(b) W5: per-tenant coverage, the right efficiency lens under fairness

Figure 8: **Residency regulation imposes no throughput tax where contention is structural, and a deliberate one where it is abusive.** (a) On W7, KVtime's goodput matches or exceeds every token meter and far exceeds the residency-blind cluster, because entitlement-aware eviction wastes less capacity on doomed re-prefills than tenancy-blind eviction. (b) On W5, the right lens is per-tenant coverage: KVtime lifts the conformant tenant from ∼0.52 to ∼0.96 while capping the rate-abusive tenant, which lowers tenant-blind aggregate goodput by construction.

What these numbers are not. Occupancy pressure is a simulator occupancy proxy, not a measured GPU SM-time utilization; a literal hardware-utilization figure requires the real-system prototype we scope as future work. KVtime's lower busy fraction (∼0.77 vs. the token meters' ∼0.91) is consistent with the goodput result—it achieves equal-or-higher goodput with less busy time because it is not thrashing on re-prefills—but we draw no hardware-efficiency conclusion the simulator cannot support.

6 Related Work

The projection lemma (lemma 1) already located each prior meter as a coordinate collapse of residency. Here we position the literature in full, through that lens: prior methods are not competitors to argue against but projections to place inside the object.

6.1 LLM serving and scheduling

Orca introduced iteration-level scheduling and selective batching for transformer serving, enabling fine-grained interleaving of requests rather than request-at-a-time execution [1]. vLLM/PagedAttention addressed KV-cache memory management by paging KV blocks, reducing waste and enabling larger effective batches [2]. Sarathi and Sarathi-Serve introduced chunked prefill and stall-free scheduling to manage the prefill/decode throughput-latency tradeoff [3, 4]. DistServe and Mooncake show that prefill, decode, and KV-cache management may be disaggregated across separate pools, exposing KV cache as a central systems object [5, 6]. FastServe and Llumnix explore preemption, rescheduling, and migration of in-memory request state [14, 15]. FairBatching studies fairness between prefill and decode tasks in stall-free batching [16]. Blink shows the scarce resource can sit off the GPU entirely: host-CPU orchestration is a critical-path bottleneck that motivates a CPU-free serving architecture [34]; in our terms this is evidence that the binding axis is context-dependent, which is precisely why the vector's axis set is open-ended (remark 4). Two recent QoS- and fairness-oriented systems sit at the scheduler/interaction layer and compose with residency rather than competing with it. Niyama co-schedules latency-class-differentiated workloads on shared infrastructure with dynamic chunking and selective relegation under overload [36]; its QoS classes are an SLO-tier concern that maps onto the urgency knob (proposition 10), orthogonal to the share/burst the residency bucket governs. FairServe

throttles users only when the KV cache is overloaded and meters with a weighted service counter at the *LLM-interaction* level rather than the request level [37]; it is token-centric and so a projection in our terms, but its interaction-level accounting is the right granularity for multi-call SLOs, a dimension our per-request residency meter does not address and the two could combine. KV compression, quantized KV, and streaming eviction policies likewise compose with rather than conflict with the meter: residency charges *measured* block occupancy, so a tenant adopting compressed or quantized KV simply holds fewer blocks and the savings pass through automatically, while a streaming eviction policy is a tiering placement in our terms.

A second line makes preemption and cross-tier KV movement cheap enough to treat as first-class scheduling actions, and in doing so exposes the very axes our vector is built to meter. ConServe drives preemption to sub-iteration, layer-wise granularity with incremental KV-cache management so that preempting and resuming a request costs near zero [30]; this realizes the low resume-cost regime our placement rule assumes and supplies the preemption/resume term of the density rule. Aqua offloads KV state across the NVLink scale-up domain with a preemptive fair scheduler, turning idle peer-GPU memory into a fast tier [35]; its interconnect path is precisely a transfer-time axis and its peer memory a tier axis in definition 4, and the externality it manages—one tenant's offloaded state consuming transfer bandwidth others need—is a residency cost on those axes that a scalar HBM-only meter cannot see, which is exactly the stock/flow blindness our transfer experiment measures (section 5.3 and proposition 9). Our framework does not compete with these systems; it provides the per-tenant currency that makes fairness across their newly cheap moves well-defined, while they provide the mechanism that makes residency on the transfer and tier axes physically realizable. Llumnix's live request migration [15] is the same composition at the cluster plane: a migration is a placement move whose cost our switching terms already price, and rescheduling-for-isolation is an execution mechanism a residency cap can govern rather than replace. FlowMesh casts multi-tenant LLM workflows as utility-based placement and batching over heterogeneous workers [38]; a residency bucket enters such a multi-objective optimizer as a per-tenant constraint, the optimizer keeping its cost/locality/throughput objective inside the fairness envelope.

A third line reconfigures capacity itself at runtime, which means the effective capacities C^r and feasible sets $\mathcal{F}(t)$ in our model are policy-driven and time-varying. Dilu co-scales GPU provisioning in two dimensions to combat fragmentation under serverless DL load [31], and MIGRator dynamically repartitions a GPU via multi-instance reconfiguration solved as an ILP [32]. Our headroom factor η and feasibility modeling are consistent with such drift, including MIG/MPS-style repartitioning—a reconfiguration event simply changes C^r and $\mathcal{F}(t)$ between epochs—but we treat capacities as quasi-stationary within an epoch and do not model the reconfiguration control loop; coupling residency entitlement to on-demand capacity is left open (section 7).

These works optimize throughput, tail latency, batching efficiency, migration, or phase-level SLOs. They do not by themselves define a tenant-level currency for long-run memory-share fairness. Our mechanism can sit above them: it decides which requests are resident/runnable; the underlying serving engine decides how to batch and execute runnable work efficiently.

6.2 Fair LLM serving

VTC defines fairness for LLM serving through a configurable service-cost function based on input and output tokens, and gives tight service-difference bounds for backlogged clients [17]. In our terms, VTC is the projection of residency onto the token axis: it meters service as a weighted token count and forgets the time those tokens are held. This is the cleanest instance of the projection lemma's time-collapse (lemma 1), and the workload of section 5 is built to expose exactly what that lost axis hides—equal token service, unequal residency. A complementary critique of the measurement side comes from Etalon, which shows that conventional latency metrics (TTFT, TBT, TPOT) incompletely capture user-perceived serving quality and proposes the fluidity index [33]; residency fairness and user-centric capacity metrics triangulate naturally, since our urgency knob is share-neutral by proposition 10 and so can chase user-facing deadlines without becoming a side channel on share.

Justitia is the closest recent work [18]. It independently identifies memory occupancy integrated over time as the right serving cost, using the scalar KV-token-time $pd + d^2/2$—the accumulative KV-cache occupation—to rank LLM applications by completion order under fair queuing, strong corroboration that memory-time is the physical primitive. Its cost is the single-axis case of residency with the prefill build-up dropped and a unit decode rate, i.e. $\pi \to \infty$ and $\delta = 1$ in C_{KV} (section 2.4); it neglects the prefill-phase

build-up $P^2/2\pi$ and the separate prefill/decode rates that the general scalar carries. We take the same primitive in two directions: we generalize the scalar to a resource *vector* across prefill/decode, tier, transfer, and instance axes; and we use residency not to rank completions but as the *currency* of a metering, fairness, and placement theory. The two papers apply the same primitive to different objectives; Justitia is recovered as the single-axis, build-up-free, unit-rate case.

Equinox blends a user-fairness counter (weighted tokens and latency) with a resource-fairness counter (throughput and GPU utilization) into a single holistic-fairness score, predicting the post-execution metrics it needs with a mixture-of-experts predictor [29]. Through the projection lens, Equinox is richer than VTC on the symptom side—it adds a latency coordinate and a utilization coordinate—but its user counter still meters weighted token service and observed latency, neither of which is the held resource: a tenant can hold large KV residency while accruing modest weighted-token and latency counts, exactly the gap proposition 2 formalizes. Its resource counter tracks aggregate utilization, not per-tenant occupancy-time, so it certifies system efficiency rather than residency isolation. Equinox is thus a two-coordinate projection (token-time symptom $\times$ aggregate efficiency) rather than a per-tenant residency meter; the projection lemma predicts it cannot certify the dominant-share guarantee, and the empirical head-to-head (section 5) confirms it: metered under our mechanism, Equinox leaves residency skewed at $\rho_{\mathrm{mt}} = 5.65$ on W7, slightly worse than VTC and far from KVtime's 4.94.

Two recent control-plane designs are close in spirit and worth positioning precisely, because each supplies a piece adjacent to ours. *Token Pools* [11] expresses tenant entitlements in inference-native units—token throughput, KV-cache bytes, and concurrency—and enforces them with debt-based, burst-aware admission and autoscaling, without modifying the runtime. This is the closest prior abstraction to a per-tenant residency entitlement, and the resemblance is not accidental: KV-cache capacity is one of its dimensions. The gap is exactly the time axis. Token Pools meters KV cache as an *instantaneous* capacity dimension—a snapshot quota, the sup-collapse of proposition 6's family—and, as its author explicitly notes, does not integrate memory held over time, identifying residency-time metering as future work. Our contribution can be read as supplying that missing dimension with formal backing: residency is the time-integral their KV-cache dimension lacks, and our per-axis buckets give the debt-based control they implement heuristically a two-sided isolation/service guarantee (theorems 2 and 3). A residency-metered Token Pool is a natural deployment of our mechanism on their control plane. *DLPM* and its distributed form D^{2}LPM [12] take a complementary concern: prefix locality. They give the first locality-aware fair scheduler, balancing cache-reuse efficiency against a deficit-counter fairness bound on token-weighted service. Their fairness unit is token-weighted and locality-aware, not residency-time, so the two concerns compose rather than compete: prefix locality enters our model through the switching costs E_r, R_r and the admissible-placement sets Φ_r (a request whose prefix is already resident has lower resume cost and a richer admissible set), while residency entitlements govern the long-run share. How a locality heuristic like DLPM's and a residency entitlement best coexist—when locality and residency fairness pull in different directions—is a concrete question our placement objective is structured to express but that we do not resolve here.

6.3 Fair queuing and multi-resource fairness

GPS/WFQ [19, 20], deficit round robin [21], MaxWeight/backpressure scheduling [22, 23], and DRF [24] provide the classical foundation for fair and throughput-aware resource sharing. The leaky/token-bucket regulators and arrival-curve service guarantees we build on for our rate-plus-burst theorems originate in deterministic network calculus [27, 28]. Our work follows this lineage but uses a different service unit: memory capacity integrated over time.

Two strands of multi-resource fair scheduling already recognize that a snapshot is not enough, and it is worth being precise about how residency differs, since the distinction is the crux of our projection lemma's treatment of DRF. THEMIS introduces *finish-time fairness* for GPU-cluster scheduling, observing that instantaneous schemes such as DRF ignore long task durations, and equalizing the factor by which sharing inflates each job's completion time [25]. Stateful DRF (SDRF) and related long-term multi-resource policies make the dominant *share* itself stateful, integrating each user's share over time and biasing allocation toward those whose time-averaged share has been low [26]. Finish-time fairness integrates the wrong object outright—the *outcome* (when a job completes), not the resource it holds; a job can finish quickly while occupying a large footprint throughout, or slowly while holding little, and finish-time fairness does not

separate these. SDRF is closer to us and deserves a careful comparison, because one could imagine simply running SDRF on KV-occupancy vectors and asking whether that already *is* our vector meter. It is not, and the reason is instructive.

Two ways to make a multi-axis residency meter. Both SDRF-on-occupancy and our vector KVtime are built from the same two ingredients—*combine the axes* into a single dominant-share number (a max over axes), and *integrate over time*—but they apply them in opposite order:

- **Combine-then-integrate (SDRF).** At each instant collapse the axes to the instantaneous dominant share $s_i(t) = \max_r k_i^r(t)/C^r$, then integrate that scalar over time: the state is $\int_0^T \max_r \left(k_i^r(t)/C^r\right) \mathrm{d}t$.
- **Integrate-then-combine (vector KVtime).** Integrate occupancy on each axis separately, $A_i^r(T) = \int_0^T k_i^r(t)\ \mathrm{d}t$, then combine once at decision time by the dominant axis: $s_i = \max_r A_i^r(T)/(C^r T)$.

These coincide only when each tenant's dominant axis never changes over the horizon; whenever the dominant axis *moves*, Jensen-style non-commutation makes the combine-then-integrate quantity the larger, $\int_T \max_r s_i^r \geq \max_r \int_T s_i^r$. In prefill/decode–disaggregated serving this is not an edge case but the norm: every request's instantaneous dominant axis flips from the prefill resource to the decode resource as it moves through its own lifecycle, and a tenant's request mix shifts over time. SDRF-on-occupancy therefore charges a tenant for being momentarily heavy on *whichever* axis is its peak at each instant—over-counting a tenant whose burden alternates between axes—whereas vector KVtime prices the sustained time-integrated load actually placed on each resource and bills the larger. The everyday reading: combine-then-integrate bills "your hardest resource at each moment, summed," while integrate-then-combine bills "your largest resource-hours total." The two agree for a tenant pinned to one axis and diverge for one whose load moves between axes.

The tradeoff. Combine-then-integrate is simpler to maintain—one scalar accumulator per tenant rather than one per axis—and it is genuinely time-aware, so it is a far stronger baseline than snapshot DRF; we include it as such, and section 5.2 confirms it is the strongest foil—tying vector KVtime on constant load and even winning long-run dominant share under axis migration, while losing the per-window isolation our buckets provide. What it gives up is twofold. First, by collapsing axes *before* integrating it loses the per-axis history, so it cannot certify a per-axis entitlement: a tenant can stay within a low time-averaged dominant share while persistently overloading one specific resource that another tenant needs, because the over-use is averaged against its light moments on the other axis. Second, and more fundamentally, SDRF is an allocation *rule* (serve the lowest-average-share tenant next), not a rate-plus-burst *regulator*; it has no bucket depth, no overdraft, and hence no two-sided arrival-curve guarantee. Our per-axis buckets are what yield the isolation and service envelopes of theorems 2 and 3, and a burst depth β an operator can tune—structure that a share-averaging rule does not possess. So SDRF-on-occupancy is the sharpest time-aware foil for our axis argument, but it is neither a special case of vector KVtime nor an equivalent of it: it integrates a collapsed scalar rather than a vector, and it regulates by averaging rather than by rate-plus-burst. Residency coincides with time-averaged share only in the degenerate constant-capacity, single-dominant-axis limit (consistent with proposition 8); the regime that dominates KV-cache serving—elastic capacity, requests whose dominant axis migrates—is exactly where the two part ways.

7 Conclusion & Future Directions

The simplest correct meter for multi-tenant LLM serving is not requests, tokens, priorities, or waiting time. It is KV memory capacity integrated over time—residency. An idealized continuous-time model renders this primitive in closed form, $P^2/2\pi + (PD + D^2/2)/\delta$—a lens that exposes its structure rather than a price the system computes; the mechanism enforces on measured occupancy throughout. From that one primitive we obtain tenant memory-share entitlements, burst buckets, preemption-aware scheduling, and multi-instance routing. The deeper point is that once serving is disaggregated, tiered, and distributed, the primitive becomes a *vector*, and that vector unifies the field: admission, preemption, routing, tiering, and prefill/decode disaggregation are one online placement problem; fairness is a dominant-share entitlement (TIDSE) on the time-integral of placement; and every prior meter is a coordinate projection that forfeits

a named guarantee. The mechanism is easy to state: each tenant is entitled to a configured fraction of dominant memory-time, and every request spends the area under its memory-time occupancy, summed over the resources it actually competes for. That is the resource the system actually runs out of. And it is measurably so: on profiled-simulator workloads where token meters certify fairness, residency entitlements cut the conformant tenant's tail latency by up to 274× at held utilization; every residency-blind meter equalizes its own coordinate while leaving residency skewed; and on axis-migration load the vector's integrate-then-combine ordering delivers the per-window isolation that even stateful time-aware DRF—the strongest prior—cannot, exactly where the theory says the two orderings must part. And on a contended spill link the stock/flow split shows its teeth: a stowaway holding half the memory of a quiet tenant monopolizes the link at nearly twice its share, a dissociation every occupancy meter is provably blind to and only the flow axis can see.

Future directions. Several questions remain open, and a few are boundaries of the model itself rather than extensions of it. *Axis separability and coupling:* our per-axis buckets assume the residency axes are separable renewable resources, but real stacks couple them—decode compute and KV bandwidth contend, fragmentation is bursty, and a control-plane or CPU-orchestration bottleneck can dominate GPU residency entirely; a bottleneck with occupancy-over-time semantics joins the vector as an axis (remark 4); one without—coupled or non-renewable—enters only as a feasibility modifier on $\mathcal{F}(t)$, and a coupled-constraint treatment is future work. *Nonstationary capacity:* we treat effective capacities C^r as quasi-stationary within an epoch, whereas reconfiguration systems (section 6) make them policy-driven and time-varying; coupling residency entitlement to on-demand capacity is open. *Online placement guarantees:* our per-epoch guarantee is the unconditional $\frac{1}{m+1}$ of theorem 8, sharpening to $\frac{1}{2}$ in the single-binding-axis regime—but it is a per-epoch approximation, not a bound across epochs. A regret or competitive analysis of the epoch *sequence* under switching costs, and a competitive bound for the cluster plane's online assignment (section 4.7), remain open. *Closed-loop stability:* we prove metering, isolation, and service guarantees, but not queue stability or a capacity-region characterization of the system in which tenants adapt their arrival policies—near saturation, bucket feasibility throttles over-entitlement tenants first while conformant backlogged tenants keep their entitled rate (theorem 3), but stability of the induced arrival dynamics is left open. *Throughput optimality:* the placement optimizer should be paired with a work-conserving batch scheduler; a heavy-traffic state-space-collapse result with residency as the workload, building on MaxWeight/backpressure, is a natural next step. *Empirical extensions:* our results are established on a profiled-microbenchmark simulator; a real-system prototype measuring low-overhead per-tenant occupancy metering and end-to-end enforcement, a placement control-plane ablation isolating the marginal value of each control, and a tiering study extending the transfer-axis dissociation to demotion across tiers are the natural next measurements, none required by the claims established here.

References

[1] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun. Orca: A distributed serving system for transformer-based generative models. In *USENIX OSDI*, 2022.

[2] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *ACM SOSP*, 2023.

[3] A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Ramjee. SARATHI: Efficient LLM inference by piggybacking decodes with chunked prefills. arXiv:2308.16369, 2023.

[4] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *USENIX OSDI*, 2024.

[5] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *USENIX OSDI*, 2024.

[6] R. Qin, Z. Li, W. He, M. Zhang, Y. Wu, W. Zheng, and X. Xu. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. In *USENIX FAST*, 2025. Also ACM Transactions on Storage; arXiv:2407.00079.

[7] Y. Cheng, K. Du, J. Yao, S. Jiang, R. Chen, S. Liu, H. Zhang, M. Geng, J. Wang, Q. Lin, and J. Jiang. LMCache: An efficient KV cache layer for enterprise-scale LLM inference (offloading and reuse across GPU, DRAM, and storage). arXiv:2510.09665, 2025.

[8] NVIDIA. NIXL: NVIDIA Inference Xfer Library—a high-bandwidth, low-latency transport for KV-cache movement across GPU, CPU, and storage. https://github.com/ai-dynamo/nixl, 2025.

[9] NVIDIA. NVIDIA Dynamo: A datacenter-scale disaggregated inference serving framework with a KV block manager routing prefill outputs to decode workers. https://github.com/ai-dynamo/dynamo, 2025.

[10] llm-d Community (IBM Research, Google, Red Hat, CoreWeave, NVIDIA). llm-d: A Kubernetes-native distributed LLM inference framework with KV-cache-aware routing and disaggregated prefill/decode serving, built on vLLM. https://llm-d.ai, 2025. CNCF sandbox project.

[11] W. J. Cunningham. Token management in multi-tenant AI inference platforms. arXiv:2603.00356, 2026.

[12] S. Cao, B. Liu, W. Zheng, S. G. Patil, T. Griggs, I. Stoica, J. E. Gonzalez, et al. Locality-aware fair scheduling in LLM serving (DLPM and D^2LPM). arXiv:2501.14312, 2025.

[13] R. Nicolae and the AtLarge Research Team. Kavier: Exploring performance, sustainability, and efficiency of LLM ecosystems through cache-aware discrete-event simulation. Vrije Universiteit Amsterdam; arXiv:2605.25247, 2026.

[14] B. Wu, Y. Zhong, Z. Zhang, G. Huang, X. Liu, and X. Jin. Fast distributed inference serving for large language models. arXiv:2305.05920, 2023.

[15] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, and W. Lin. Llumnix: Dynamic scheduling for large language model serving. In *USENIX OSDI*, 2024.

[16] H. Lyu, B. Liu, M. Wu, and H. Chen. FairBatching: Fairness-aware batch formation for LLM inference. arXiv:2510.14392, 2025.

[17] Y. Sheng, S. Cao, D. Li, B. Zhu, Z. Li, D. Zhuo, J. E. Gonzalez, and I. Stoica. Fairness in serving large language models. In *USENIX OSDI*, 2024.

[18] M. Yang, G. Wang, M. Luo, Y. Liu, C. Chen, H. Zhao, Y. Feng, Q. Chen, and M. Guo. Justitia: Fair and efficient scheduling for LLM applications. arXiv:2510.17015, 2025.

[19] A. K. Parekh and R. G. Gallager. A generalized processor sharing approach to flow control in integrated services networks: The single-node case. *IEEE/ACM Transactions on Networking*, 1(3):344–357, 1993.

[20] A. Demers, S. Keshav, and S. Shenker. Analysis and simulation of a fair queueing algorithm. In *ACM SIGCOMM*, 1989.

[21] M. Shreedhar and G. Varghese. Efficient fair queuing using deficit round robin. In *ACM SIGCOMM*, 1995.

[22] L. Tassiulas and A. Ephremides. Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks. *IEEE Transactions on Automatic Control*, 37(12):1936–1948, 1992.

[23] A. L. Stolyar. MaxWeight scheduling in a generalized switch: State space collapse and workload minimization in heavy traffic. *Annals of Applied Probability*, 14(1):1–53, 2004.

[24] A. Ghodsi, M. Zaharia, B. Hindman, A. Konwinski, S. Shenker, and I. Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In *USENIX NSDI*, 2011.

[25] K. Mahajan, A. Balasubramanian, A. Singhvi, S. Venkataraman, A. Akella, A. Phanishayee, and S. Chawla. Themis: Fair and efficient GPU cluster scheduling. In *USENIX NSDI*, 2020.

[26] L. Suresh, et al. Stateful dominant resource fairness: Considering the past in a multi-resource allocation. Technical report / arXiv, 2018.

[27] R. L. Cruz. A calculus for network delay, Part I: Network elements in isolation. *IEEE Transactions on Information Theory*, 37(1):114–131, 1991.

[28] J.-Y. Le Boudec and P. Thiran. *Network Calculus: A Theory of Deterministic Queuing Systems for the Internet*. Springer LNCS 2050, 2001.

[29] Z. Wei et al. Equinox: Holistic fair scheduling in serving large language models. arXiv:2508.16646, 2025.

[30] Y. Qiao, S. Anumala, J. Yang, et al. ConServe: Fine-grained GPU harvesting for LLM online and offline co-serving. arXiv:2410.01228, 2024.

[31] B. Yu, et al. Dilu: Enabling GPU resourcing-on-demand for serverless DL serving via introspective elasticity. In *ASPLOS*, 2025.

[32] Y. Xu, et al. Improving GPU multi-tenancy through dynamic multi-instance GPU reconfiguration. arXiv:2407.13126, 2024.

[33] A. Agrawal, A. Agarwal, N. Kedia, J. Mohan, S. Kundu, N. Kwatra, R. Ramjee, and A. Tumanov. Etalon: Holistic performance evaluation framework for LLM inference systems. arXiv:2407.07000, 2024.

[34] M. Siavashi et al. Blink: CPU-free LLM inference by delegating the serving stack to GPU and SmartNIC. arXiv:2604.07609, 2026.

[35] A. Vijaya Kumar et al. Aqua: Network-accelerated memory offloading for LLMs in scale-up GPU domains. In *ASPLOS*, 2025. arXiv:2407.21255.

[36] K. Goel, J. Mohan, N. Kwatra, R. S. Anupindi, and R. Ramjee. Niyama: Breaking the silos of LLM inference serving. arXiv:2503.22562, 2025.

[37] R. Sheng et al. Ensuring fair LLM serving amid diverse applications (FairServe). arXiv:2411.15997, 2024.

[38] FlowMesh authors. FlowMesh: A service fabric for composable LLM workflows. arXiv:2510.26913, 2025.

[39] G. B. Dantzig. Discrete-variable extremum problems. *Operations Research*, 5(2):266–288, 1957.

[40] H. Kellerer, U. Pferschy, and D. Pisinger. *Knapsack Problems*. Springer, 2004.

[41] A. Kulik and H. Shachnai. There is no EPTAS for two-dimensional knapsack. *Information Processing Letters*, 110(16):707–712, 2010.

[42] X. Han, Y. Kawase, and K. Makino. Randomized algorithms for online knapsack problems. *Theoretical Computer Science*, 562:395–405, 2015.

A Closed-Form Fidelity: the κ-Bound

The closed form assumes phase-pure prefill and decode at fixed rates π, δ. Real serving violates both assumptions: throughput depends on the instantaneous batch composition and on sequence length (attention cost grows with context), and chunked-prefill schedulers such as Sarathi-Serve interleave prefill and decode within an iteration [4]. We do not need constant throughput for the *meter* to be correct; the meter is defined as the area under the realized KV-residency curve (definition 1). What can drift is the convenience of the *closed-form price* as a predictor of that area. We bound this drift.

Definition 9 (Realized vs. nominal KV-time)**.** *Let $C_{\mathrm{KV}}(P, D)$ be the closed-form price and let $\widetilde{C_{\mathrm{KV}}}(P, D) = \int_0^{\widetilde{T}} k(t)\,\mathrm{d}t$ be the area under the actually realized residency curve, where $\widetilde{T}$ is the realized completion time under contended, batch-dependent throughput.*

Proposition 12 (Residency area is rate-invariant; only timing rescales)**.** *Suppose the KV-token* trajectory in token space *is unchanged—prefill monotonically loads P tokens, then decode adds one token per generated step up to D—but the per-step wall-clock durations vary, with the prefill step rate lying in $[\pi_{\min}, \pi_{\max}]$ and the decode step rate in $[\delta_{\min}, \delta_{\max}]$. Then*

$$\frac{\pi}{\pi_{\max}} C_{\text{pre}}(P) + \frac{\delta}{\delta_{\max}} C_{\text{dec}}(P, D) \;\le\; \widetilde{C_{\text{KV}}}(P, D) \;\le\; \frac{\pi}{\pi_{\min}} C_{\text{pre}}(P) + \frac{\delta}{\delta_{\min}} C_{\text{dec}}(P, D),$$

where $C_{\text{pre}}, C_{\text{dec}}$ are the nominal phase costs from the closed-form theorem evaluated at the reference rates π, δ.

Proof. The residency level after processing p prompt tokens is p regardless of how fast those tokens are processed; the KV-time contribution of the infinitesimal mass $\mathrm{d}p$ is $p \cdot$ (time to process $\mathrm{d}p$). If the realized prefill rate at that point is $r \in [\pi_{\min}, \pi_{\max}]$, the time is $\mathrm{d}p/r \in [\,\mathrm{d}p/\pi_{\max},\, \mathrm{d}p/\pi_{\min}]$. Integrating the residency level p against these bounds and factoring out the reference rate π gives the prefill bracket; the identical argument with residency level $P + d$ gives the decode bracket. Summing the phase brackets yields the claim. □

Corollary 3 (Multiplicative accounting error is bounded by throughput variation)**.** *If realized phase rates stay within a factor $\kappa \ge 1$ of their reference values, i.e. $\pi/\kappa \le r \le \kappa\pi$ and $\delta/\kappa \le r \le \kappa\delta$, then*

$$\kappa^{-1} C_{\text{KV}}(P, D) \le \widetilde{C_{\text{KV}}}(P, D) \le \kappa\, C_{\text{KV}}(P, D).$$

Thus the closed-form price is accurate to within the multiplicative throughput-variation factor κ, uniformly over (P, D).

Proof. Specialize proposition 12 with $\pi_{\min} = \delta_{\min} = (\cdot)/\kappa$ and $\pi_{\max} = \delta_{\max} = \kappa(\cdot)$, then add the two phase brackets and factor. □

The bound's role is to certify the *idealization*, not the mechanism. The entitlement and bucket machinery never relies on the closed form: buckets drain on *measured* $k_i(t)$, so every fairness and service guarantee (section 3.1) holds for the realized meter regardless of throughput dynamics, and no claim in the paper depends on the formula's numerical accuracy. What corollary 3 establishes is that the idealized lens is itself faithful: the bound is multiplicative and symmetric—realized residency lies within a factor κ of the nominal C_{KV} whenever per-request throughput stays within κ of the fitted rates—so the structural facts the lens reveals (quadratic prefill build-up, prefill/decode separation) are stable under realistic rate variation, with speculative decoding and dynamic batching entering only through the realized rates and absorbed into the same κ. This is model-robustness, a property of the idealization, not a validation the mechanism requires. When chunked prefill blends phases, the same area integral applies with the token-space trajectory replaced by the realized interleaving, carrying an additional cross term in the decomposition; since the meter integrates measured occupancy regardless, this affects only the lens's structural decomposition, not what is charged.

B A Practical Variant: Discounted (Recency-Weighted) Utilization

The main fairness development uses the undiscounted leaky-bucket balance, on which the two-sided guarantees of theorems 2 and 3 are proved. In practice an operator may prefer fairness that forgets old usage, so that a tenant idle for a long stretch is not credited indefinitely. We record the natural recency-weighted variant here; it is an engineering knob, not relied on by any result in the main text, and we do not claim the arrival-curve guarantees transfer to it unchanged (the EWMA debt below is a low-pass filter rather than a token bucket, so its guarantees would require a separate derivation).

Definition 10 (Discounted KV utilization)**.** *For discount rate $\alpha > 0$, tenant i's discounted KV utilization at time t is*

$$D_i^{\alpha}(t) = \alpha \int_0^t e^{-\alpha(t-s)} \frac{k_i(s)}{K}\, \mathrm{d}s.$$

Equivalently,

$$\dot{D}_i^\alpha(t) = \alpha\left(\frac{k_i(t)}{K} - D_i^\alpha(t)\right).$$

Tenant i is below discounted entitlement when $D_i^\alpha(t) < \omega_i$.

A discounted scheduler may replace $B_i(t)$ in eq. (1) with $\omega_i - D_i^\alpha(t)$ or with a bucketed function of this EWMA debt. Discounting does not change the physical request charge; it changes how quickly the scheduler forgives old usage.

C The Idealized Leximin Allocation Rule and Its Recovery of DRF

The main text enforces a dominant-share *cap* (TIDSE, definition 7), which is what the mechanism implements and what all guarantees concern. For completeness we record the idealized allocation *rule* it is inspired by, and the precise sense in which that rule generalizes classical DRF. Nothing in the main development depends on this; computing the rule exactly is not part of the deployed mechanism, and an exact leximin scheduler is left as an extension (section 7).

Definition 11 (Leximin time-integrated DRF)**.** *A placement policy is* leximin time-integrated dominant-resource fair *if the long-run dominant shares $\{s_i\}$ are leximin-optimal subject to feasibility: the sorted vector of dominant shares is lexicographically minimal, so no tenant's dominant share can be lowered without raising another tenant's already-equal-or-higher share. Like classical DRF, this is an allocation* rule *(it prescribes the allocation), in contrast to the enforced cap of definition 7, which only bounds each tenant's share.*

Proposition 13 (Leximin TI-DRF recovers classical DRF under constant occupancy)**.** *If each tenant's occupancy vector $\mathbf{k}_i(t) \equiv \mathbf{k}_i$ is constant in time, the leximin TI-DRF rule reduces to classical dominant resource fairness [24] on the static demand vectors $\mathbf{k}_i$.*

Proof. With $\mathbf{k}_i$ constant, $\mathbf{M}_i(T) = \mathbf{k}_i T$ and the dominant share $s_i = \max_r k_i^r / C^r$ is time-independent. Making the sorted vector of these static dominant shares lexicographically minimal subject to feasibility is exactly the dominant-resource-fair allocation of Ghodsi et al. on the demand vectors $\mathbf{k}_i$. □

This is the fairness-side analogue of the metering recoveries of section 2.4: the idealized rule contains classical DRF as its constant-occupancy, allocation-rule limit, while the enforced cap we actually deploy contains the static dominant-share entitlement as its limit (proposition 8). The two coincide when entitlements are set proportional to a common target; in general the cap is the implementable surrogate for the rule. The priority/SLO structure of the main text (share/burst/urgency) is naturally expressible on the cap but not on a pure leximin rule—an egalitarian allocation rule has no separate urgency dimension—which is one practical reason the deployed mechanism is the cap rather than the rule.

D Appendix: Scalar Special Case of the Placement Algorithm

For reference, the scalar ($m = 1$) special case of PlaceKV (section 4.3) is the following entitlement scheduler; it is the form used in the single-axis development of section 3.1 and is recovered from eq. (2) by taking one HBM axis and unit urgency.

E Appendix: A Minimal Separation from Linear Token Pricing

The prefill-only separation in the main text is intentionally simple. It shows that when a tenant controls request shaping, linear token counters cannot distinguish one long resident request from many short resident requests. Proposition 2 gives the non-degenerate version: even with equal mean length and equal request count, length *variance* alone drives an unbounded KV-time gap that a linear meter cannot see. In practice, request overheads, batching granularity, and prefix reuse will change the constants. They do not remove the structural point: KV pressure is an area, not a count.

Algorithm 2 KV-Time Entitlement Scheduler (scalar special case of PlaceKV)

Configure tenant shares ω_i, bucket depths $B_i^{\max}$, overdrafts H_i, headroom η.
for each scheduling epoch t **do**
  Update tenant usage $A_i(t) \leftarrow A_i(t^-) + \int_{t^-}^{t} k_i(s)\,\mathrm{d}s$.
  Update bucket balances $B_i(t) \leftarrow \min\{B_i^{\max}, B_i(t^-) + \omega_i K(t - t^-) - [A_i(t) - A_i(t^-)]\}$.
  Build candidate set $\mathcal{R}(t)$ from waiting, resident, and resumable requests.
  Build feasibility family $\mathcal{F}(t)$ from KV capacity, fragmentation, and placement constraints.
  Estimate progress density $\rho_r(t)$ and switching/resume costs $E_r(t), R_r(t)$.
  Choose $S^\star(t)$ by eq. (1).
  Admit requests in $S^\star(t) \setminus S^-(t)$; continue requests in $S^\star(t) \cap S^-(t)$; preempt requests in $S^-(t) \setminus S^\star(t)$.
end for

F What the Residency Abstraction Buys

Deployment. The mechanism requires no change to existing clients. OpenAI-compatible tenants submit requests exactly as today; the platform observes what it already has—prompt and output lengths (P, D), realized KV occupancy over time, cached-prefix residency, and instance/tier placement—and drains each tenant's memory-time buckets automatically from the measured occupancy (definition 2). Entitlements ω_i and bucket depths are set per service contract. Optional APIs expose an urgency/SLO class and a max-token budget, but default policies (uniform urgency, contract-derived ω_i) work without any client awareness of the underlying mechanism. The scheduler is the unified selector of section 4.3, which most serving stacks can host as the admission/preemption/routing policy above an unchanged batch executor.

We do not claim prior LLM schedulers are wrong; we claim they optimize a different layer. Orca, vLLM, Sarathi-Serve, DistServe, Mooncake, Llumnix, FastServe, and FairBatching improve execution, batching, memory management, prefill/decode separation, or migration. VTC and Justitia improve fairness at the scheduler/application layer. KV-time entitlements address a complementary governance question: how should a multi-tenant platform meter, budget, and schedule the actual scarce resource across repeated tenant interactions?

Table 4 summarizes the comparison. The abstraction has four properties:

1. **Physical:** the charge is the time-integral of a request's placement across the resources it actually competes for.

2. **Fair:** entitlements cap dominant memory-time share (TIDSE, a DRF-inspired entitlement), recovering per-axis rate-plus-burst isolation.

3. **Composable:** admission, continuation, preemption, routing, tiering, and PD disaggregation are one feasible-placement problem (theorem 6).

4. **Reductive:** prior meters are coordinate projections of this object, each forfeiting a named guarantee (lemma 1).

What is proved, assumed, and left to measurement. To help a reader not over-read the theory, table 5 maps each headline claim to its result, its key assumption, and whether its confirmation is analytical or empirical.

Mechanism	Service unit	Structural limitation
Token-WFQ / VTC [17]	input + output tokens	misses PD/δ residency term
Decode-token fairness	output tokens only	ignores prompt residency during decode
Request-count sharing	one count per request	no size or duration awareness
AWT / delay-based	realized waiting time	load-dependent symptom, not cause
Instantaneous KV quotas	peak $k_i(t)$	ignores duration of occupancy
Static DRF over GPU/CPU	resource vector	no time-varying KV trajectory
Justitia [18]	KV token-time (per app)	application-level, not tenant-repeated
Scalar KV-time (Part one)	KV-token-seconds	single axis; no PD/tier/route fairness
This paper	**memory-time vector**	**dominant-share entitlement (TIDSE); unifies all rows**

Table 4: Service units used by candidate fairness mechanisms for LLM serving, and what each misses relative to KV-time entitlements.

Claim	Result	Key assumptions	Status
Closed-form KV-time price	theorem 1	phase-pure progress; constant π, δ	proved; fidelity bounded (corollary 3)
Idealized lens robust to rate variation	corollary 3	variation factor κ	proved (model-robustness)
Isolation (upper bound)	theorem 2	per-axis bucket enforced	proved
Service / no starvation (lower bound)	theorem 3	work-conserving; $\sum \omega_j \leq \eta$; fluid (entitlement-saturating) backlog; $\tau_{\text{idle}} = o(T)$	proved; rate measured (H2)
Prior meters are projections	lemma 1	stated collapse maps	proved
Distributional separation	proposition 2	equal mean/count, differing variance	proved; shown (H3)
Placement unifies ops	theorem 6	axis/menu encoding	proved
Greedy $\frac{1}{2}$ guarantee	theorem 7	single binding axis	proved
Multi-axis greedy $\frac{1}{m+1}$	theorem 8	m axes; reduction to m-dim. knapsack	proved; ablation scoped
Urgency-invariant isolation	theorem 4	bucket-feasible selection	proved
Share-neutral priority	proposition 10	self-declared urgency	proved (share only)

Table 5: Claim matrix: each headline claim, the formal result, its key assumptions, and whether it is proved, cited, heuristic, or to be confirmed empirically (hypothesis labels refer to section 5).